Engineering an Agentic Network Operations Platform

Findings from Nearly Two Years of Development, Laboratory Evaluation, and Field Use

Alex Cronin

Abstract. **We report an empirical study of the troubleshooting capability of an agentic network operations (NetOps) platform developed over approximately twenty-two months into a multi-platform system with interactive and alert-triggered investigations. A retrospective census identifies 914 command-line investigation captures representing 913 conversations and 118 alert-harness results, separate from the units of five focused experiments. In a compatibility experiment of 260 planned device–query cells and 256 executed responses, every execution reports normal transport return while 19 are rejected or incomplete. A controlled configuration experiment compares eight matched-MTU investigations with ten symptom-prompt variants after one MTU intervention: all ten modified-state answers identify the mismatch with both-end evidence, and none executes an active dataplane verification. In a 60-investigation cohort, 14 rejected tool returns across seven investigations are labeled successful and all seven evaluator error counts are zero; matched-prompt follow-ups show explicit failure payloads preceding correct failure presentation. Matched device–query pairs from two execution revisions show median command latency falling from 12.15 to 4.60 seconds while the request-to-completion interval falls from 81.74 to 64.18 seconds. A bounded field evaluation adds audit-recorded cases of unsupported endpoint attribution and detail lost in summarized context. We formalize claim support, measurement projections, and lifecycle observables to show why command completion, anomaly detection, diagnostic causality, evaluator verdicts, and operator delivery require different evidence, and we derive validation requirements from the gaps. The contribution is quantitatively reconstructed engineering findings, not an aggregate accuracy, productivity, or MTTR estimate. Anonymized measurement tables and a verification script accompany the paper; customer identities and implementation details remain private.**

Index Terms. **agentic NetOps; network troubleshooting; mean time to repair (MTTR); large language model agents; diagnostic evidence; empirical evaluation; evaluation validity; latency decomposition**

I. Introduction

Network troubleshooting starts from uncertainty. A user may report intermittent connectivity without knowing which interface is affected. An alert may describe a symptom that has already cleared. Device output may mix useful observations with unsupported commands, stale events, and facts about unrelated objects. Whoever troubleshoots, engineer or agent, has to decide what to inspect, work out what each result means, and establish which relationships connect those results to the original problem.

Our system is an agentic network operations (NetOps) platform, in the sense industry analysts now give the term: AI agents that sense, reason, and act on network tasks within defined guardrails [31], [32]. It is built for troubleshooting, because the cost of an unresolved incident is measured in mean time to repair (MTTR) and reducing MTTR is the platform's objective; in the evaluated scope, acting means bounded, read-only device queries (Section X). This paper studies the platform's troubleshooting function: it answers a network question by selecting devices and tools, collecting operational observations, and generating an evidence-based response, and it supports interactive requests, alert-triggered work, and follow-up investigations, so the operator does not have to translate every question into a sequence of device commands. It does not measure MTTR; Sections VIII and IX state why the reported intervals are not MTTR estimates. The engineering problem is not exhausted by connecting a language model to a command interface. A request can finish normally while returning a rejected command. A real observation can concern the wrong target. A correct configuration finding can be accompanied by an unsupported explanation of traffic impact. [E7, E8]

The development history begins with an integrated monitoring and troubleshooting implementation and extends through multi-device execution, platform integration, evidence retention, execution controls, command-line interaction, and evaluation. Source history establishes approximately twenty-two months of engineering; Section II states what that span does and does not establish. [E1, E7]

The paper first documents that development trajectory (Section III) and then asks three concrete questions.

RQ1 Which observable success signals disagree with the evidence they appear to summarize?

RQ2 Which recorded engineering changes demonstrably improve a defined behavior, and which downstream conditions remain unchanged?

RQ3 How much of an observed command-latency improvement survives into the request-to-completion-stamp interval?

Our answer is that correctness must be evaluated at the boundary relevant to the claim. Figure 1 summarizes those boundaries and the observed counterexample at each. We establish this through five quantitative components: a fully enumerated platform-probe population; a controlled MTU state contrast; a raw-return and evaluator disagreement cohort with staged correction records; a matched-command execution-refinement sequence; and process and answer consistency checks across

A. Cronin is with Nanites AI. Preprint, not peer reviewed. September 2026.

repeated baseline and alert campaigns. A bounded external field evaluation supplies audit-recorded qualitative cases that the laboratory experiments do not reach, in particular target binding under failed endpoint lookups and evidence handling across summarized context. The mathematical treatment defines what each measurement can identify. It does not describe hidden production schemas or claim a new general theorem.

The work is original to this engineering program: the implementation, captured investigations, validation campaigns, revisions, and failure analyses are our own. "Original" refers to provenance, not to a claim that every principle discussed was first invented here. The paper makes no claim that an unspecified architecture is globally novel or superior to other systems, and it uses no results from other systems to support its own; Section XIII positions the findings against published work. Its contributions are specific observed failure relationships, verified stage-level corrections, a quantitative performance decomposition supported by retained artifacts, and a set of validation requirements derived from the observed gaps.

II. Retrospective Corpus, Evidence Classes, and Measurement Protocol

A. Scope and anonymization

The inspected source history covers about twenty-two months, from the first functional implementation to the latest retained revision; we call this span "nearly two years." The machine-censused evaluation archive is narrower: its identified request-start events cover approximately ten weeks within that history. The records form a longitudinal development record. They do not establish continuous staffing, continuous production operation, or a controlled experiment lasting two years, and the archive is a convenience sample of preserved engineering work, not a prospectively sampled population of network incidents. [E1, E7]

The evidence comes from our development environment, laboratory work, and one bounded external evaluation. The laboratory and the external environment differed in hardware, software, access arrangements, and the availability of fault ground truth. Some tests used retained outputs or isolated components; others used live device interfaces. We identify these settings separately wherever the distinction affects an inference, and we do not merge command probes, conversation turns, repeated scenarios, or component tests into one experimental population.

Anonymization removes customer and product identities, actual addresses, interface names, exact calendar dates, platform combinations, device inventories, topology, and original run handles. Device, query, and campaign aliases retain within-experiment relationships. Numerical observations are not fabricated or perturbed to disguise results. Proprietary prompts, schemas, source code, orchestration logic, implementation contracts, and tuning parameters are not released; general intervention categories and measured outcomes are disclosed because they are necessary to evaluate the paper's claims. The paper is an experience report with a reproducible numerical audit, not a reproducible specification of the proprietary system.

B. Selection, provenance, and observation units

A bounded offline extraction pass processed 2,439 selected textual and structured source files, totaling 94,882,450 bytes. A separate pass hashed 5,907 retained raw-output files totaling 44,552,729 bytes. The source index records each file's byte size, line count, and SHA-256 digest, and extracted observations carry the original line or structured-field location. Existing files were not changed, and no new device experiment or service query was performed. The selected-file passes completed without parse errors or cap-triggered omissions. This establishes extraction coverage, not semantic adjudication of every answer. [E1]

Selection included retained evaluation, targeted-check, smoke-test, and replay-support records. It excluded unrelated security material, imported bundles, earlier manuscript artifacts, and nonselected documentation collections. Recognition used explicit transcript identity headers and request-event records; directory names alone did not establish an executed trial. Test definitions, generated inventories, command probes, grading rows, conversation transcripts, and individual tool outputs remain different observation units. Thousands of tool-output files therefore cannot be reported as thousands of independent evaluations. [E1]

The resulting CLI corpus contains 914 transcript files with distinct file hashes, representing 913 conversation identities (Figure 2). One conversation contains separately captured initial and follow-up requests; these are two invocations, not a duplicate transcript. Four driver indexes repeat 33 already-represented identities and contribute no additional invocation. Nine alert driver logs are also excluded from the CLI count. Separately, 118 alert-result files have distinct hashes but reuse 42 conversation identities. Their associated transcripts can contain earlier turns, so counting every visible completion would overcount newly evaluated requests. [E1]

For corpus description, identity deduplication operates at the conversation level while retaining separate invocation timestamps. For each experiment, the denominator comes from its own manifest or structured result population, and the broad census is not pooled into a single accuracy estimate. Source-directory provenance divides the 913 CLI conversations into 759 batch-evaluation records, 28 targeted records, 11 one-off records, and 115 other engineering checks; these labels do not imply randomization or statistical independence. [E1]

C. Evidence classes and claim verification

Beyond the machine census, we used historical reports to locate candidate findings, then checked selected claims against retained artifacts and against the relevant implementation or evaluator code (Figure 3). The full internal post-mortem was reviewed as a secondary synthesis. Where it referred to correspondence, assessment slides, or transcripts that were not available, those materials were not treated as independently inspected sources, and the relationship and commercial narratives in that document are not empirical findings of this paper. Table I distinguishes the evidence classes and their limits. A current test definition shows what a test is designed to check; it is not counted as a newly executed test.

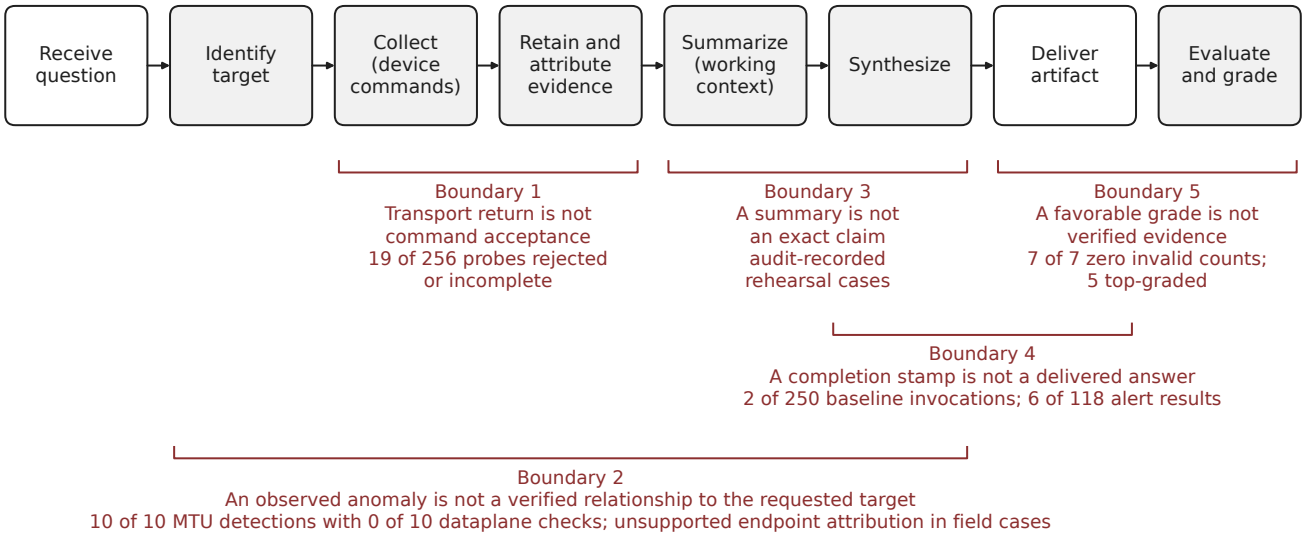


Fig. 1. Stages of a tool-using network investigation and five boundaries at which a success signal disagreed with the underlying evidence. Each bracket names the failed implication and the observed counterexample. Boundary 1 is documented in Sections V and VII, Boundary 2 in Sections VI and X-A, Boundary 3 in Section X-C, Boundary 4 in Section IX, and Boundary 5 in Section VII. Boundaries 1, 4, and 5 are counted from laboratory and harness records; Boundary 2 combines a counted laboratory result with audit-recorded field cases; Boundary 3 is audit-recorded only. A label at one boundary does not validate the next.

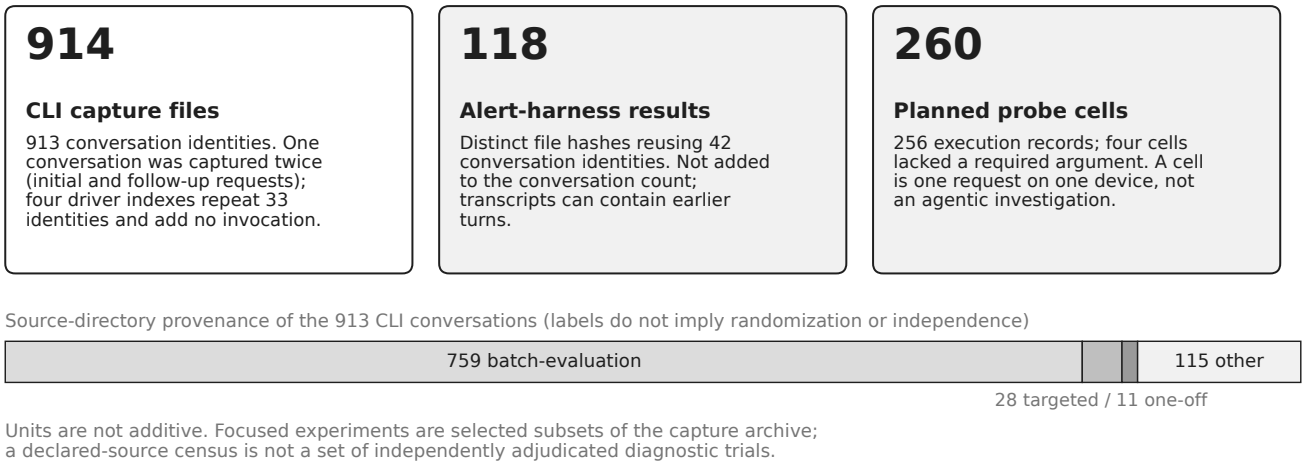


Fig. 2. Distinct units in the retained evidence. The counts are not additive: focused experiments overlap the capture archive, alert results reuse conversation identities, and a command-probe cell is not an agentic investigation. The provenance bar partitions the 913 CLI conversations by source directory.

Cases outside the five quantitative experiments were chosen to examine consequential mechanisms and to include both positive and negative observations. This is purposive selection, not random sampling, exhaustive incident coding, or a prevalence study. For each such finding we separated the requested outcome, the evidence actually available, the interpretation made from that evidence, and the status of any later correction, and we looked for contradictions between summaries and underlying records, between successive versions of a document, and between a claimed validation and the behavior the test actually exercised. In this paper, "directly inspected" means retained artifacts or source examined during preparation. "Audit-recorded" means conclusions documented in earlier internal investigations.

This process turned up inaccuracies even in earlier corrective reports. Some reported totals did not reconcile with the structured records, and one apparent live validation was later narrowed because the intended behavior had not been exercised (Section XI-C). We keep such discrepancies as evidence about the evaluation process rather than resolving them in favor of the most favorable summary. [E2, E4, E10]

D. Repeated prompts and measurement boundaries

The repeated-baseline subset contains ten reported campaigns, each with 25 observations: 250 distinct conversation identities and 25 byte-identical prompt strings repeated across

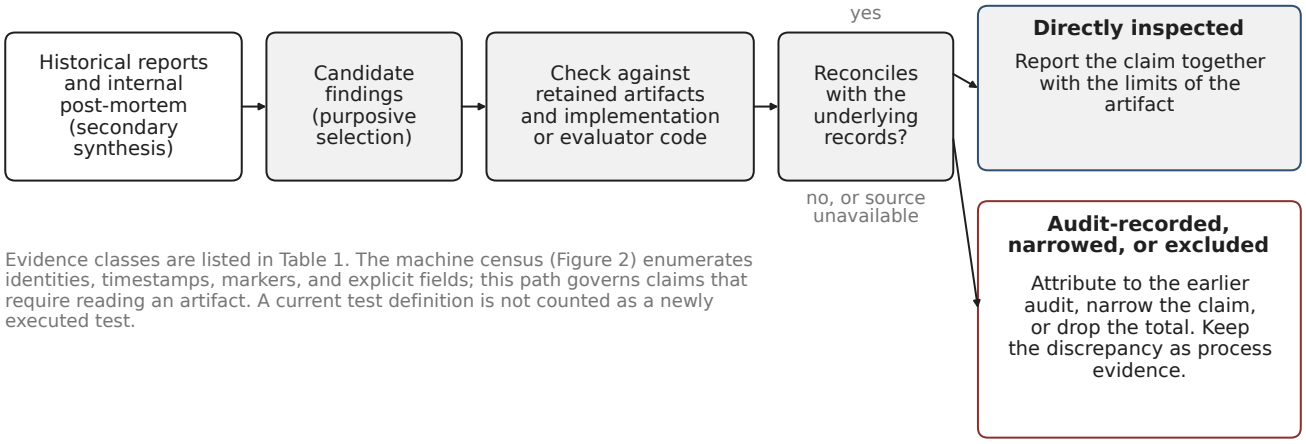


Fig. 3. Claim verification path for findings that require reading an artifact rather than counting a field. Claims that reconciled with the underlying records are reported as directly inspected. Claims that did not reconcile, or whose sources were unavailable, are attributed to the earlier audit, narrowed, or excluded, and the discrepancy is kept as evidence about the evaluation process.

TABLE I
EVIDENCE CLASSES AND THEIR INTERPRETIVE LIMITS.

Evidence class	Use in this study	Limit
Source history and implementation	Establish capabilities present at a recorded revision and trace selected changes	Does not establish deployment, enablement, or operational success
Retained command output and execution captures	Verify what a preserved observation or execution record contains; supply the machine-censused populations	May lack the full environment, event history, or collection context
Investigation and validation audits	Reconstruct broader trajectories and reported outcomes	Audit-recorded findings are not equivalent to newly retrieved raw traces
Evaluation definitions and result records	Examine what was measured and how results were assigned	Labels depend on the evidence available to the evaluator
Design and planning documents	Establish intended direction and reasons for proposed work	A proposal is not an implemented capability

campaigns. Every result row was joined to an existing transcript with a matching identity. Three additional incomplete campaigns contain two captures each but no final report; those six captures remain in the broad census and are excluded from the 250-observation subset. [E6]

For baseline invocation i, the recorded processing interval is

$$\tau_i = t_{i,\text{completion-stamp}} - t_{i,\text{request-received}}.$$

Both endpoints come from transcript event timestamps. The interval includes request processing and ends at the process-completion stamp; it is not necessarily time to a successfully rendered answer. Missing endpoints are not replaced by zero. Throughout the paper, elapsed-time statistics and percentage changes are computed from retained-precision event timestamps, and rounding is applied only for presentation; recomputing a statistic from displayed values can differ in the last digit, for example 124.85 rather than 124.8 seconds for the MTU median in Section VI-C and 53.75 rather than 53.74 percent for the collection-interval reduction in Section VIII. Campaign summaries retain failed and timed-out observations when both endpoints exist. The latest source tree and a transcript's model header do not establish an immutable historical code snapshot, so changes in implementation, model roles, device state, and concurrent work limit causal interpretation of cross-campaign differences. [E1, E6]

E. Focused experimental populations

Table II lists the declared population and primary question for each quantitative evidence set. Rows are not summed into a grand experimental N.

F. Researcher perspective

Most of the underlying records were produced by the system's engineering team, often with model assistance. The causal-review labels in Section VI and the earlier internal audits were produced with language-model assistance and have not been independently adjudicated; a blinded human review packet exists but had not been completed at the time of writing. No preregistered rubric, independent human adjudication, or inter-rater reliability study was conducted for this paper. Language-model tools were also used in preparing the manuscript itself, as stated in the acknowledgments. The provider's perspective, selective preservation of evidence, and shared assumptions across model-assisted reviews are possible sources of bias.

TABLE II
DECLARED POPULATIONS AND THE CLAIM EACH EVALUATES.

Evidence set	Population	Primary question	Evidence review
Compatibility probes	260 planned cells; 256 executions	Transport versus command acceptance	All original rows; all 247 nonempty bodies retained and scanned
MTU state contrast	8 preceding and 10 modified-state investigations	Detect the actual configuration mismatch	All 18 answers reviewed; both-end observations recalled
Evaluator disagreement	60 investigations; 510 delimited tool-output previews	Does a favorable label preserve raw rejection evidence?	All declared previews and grade rows scanned; selected events inspected
Correction follow-ups	7 reruns and 2 targeted invocations	Which correction stage is demonstrated?	All 14 repeated rejection events; targeted checks
Execution refinement	5 retained runs; 95 device calls	Matched-call versus lifecycle latency	All execution, result, and phase-boundary events
Repeated baseline	10 campaigns of 25 prompts	Process outcome and recorded interval	All 250 report and transcript joins
Alert harness	118 results, including 16 follow-up blocks	Current-invocation completion and answer presence	All structured records; targeted disagreement traces

III. DEVELOPMENT TRAJECTORY

A. From an integrated prototype to broader execution

The first implementation connected monitoring and alerting with an interactive troubleshooting application and device operations. It provided paths for an operator request or a monitoring event to start a tool-using investigation and return an explanation, and it already recognized that identifying permitted devices and obtaining authorization for configuration work mattered. The source history alone does not establish that every early path executed successfully, that platform compatibility was broad, or that performance under partial failure was dependable. [E7]

Development then widened the range and organization of device interactions, the handling of concurrent investigations, and the continuity of state across background execution. The system gained support for broader command surfaces and platform-specific behavior. That increased the kinds of questions it could attempt and exposed differences that a working connection alone could not resolve: a command may run over a supported transport and still be invalid for a device release, return a valid result that lacks the field the investigation needs, or return relevant data only after another query has identified the correct target. These differences became part of the engineering problem rather than exceptions outside it; Section V measures them directly. [E2, E7]

B. Investigation quality as a system property

As investigations spread across devices and turns, more failures involved the relationship between components that each worked on its own. Useful evidence could be collected and then left out of the next decision. A missing observation could be described honestly but left uncollected despite an available path. A later answer could repair an unsupported intermediate claim while leaving the trajectory confusing or inefficient. The engineering response included stronger support for retaining evidence, associating observations with their sources, checking unresolved requirements, and making execution outcomes inspectable. The observable change in emphasis was from asking whether the assistant could call a tool to asking whether the whole investigation justified the answer delivered to the operator. [E8, E9, E10]

C. Reusable validation and the resulting capability set

Later records show more explicit platform onboarding, retained command fixtures, replay-oriented evaluation, and distinctions between process completion and answer quality. They also show experimental features, rejected approaches, reverted changes, and validation that remained incomplete. Progress was iterative, and different subsystems reached different levels of maturity. Table III and Figure 4 summarize the stages at a level that does not disclose implementation mechanics. The stages overlap, are not discrete releases, and do not imply that earlier problems were resolved before later work began. [E2, E12, E13]

The resulting implementation could access live device observations, configured inventory and topology, and historical telemetry where a platform supported it. Inventory establishes what the application knows about the environment; a live query establishes what a device returned at collection time; a time series can expose change that a single snapshot cannot. Implementing access to all three did not establish that every investigation used them or that historical telemetry was available for every platform. Operator-facing work included interactive and command-line access, follow-up investigation, and findings presented as narrative reports, tables, and diagrams. A well-rendered report or a useful topology view was still an artifact to validate, not independent evidence that its underlying observations or relationships were correct. [E8, E12]

IV. AN ANALYTICAL MODEL OF EVIDENCE AND SUCCESS

A. Claims, observations, and target binding

For an investigation i, let E_i be the retained observations and K_i a finite, nonempty set of evidentiary requirements for a stated claim. An observation is described analytically by its

TABLE III
OBSERVABLE DEVELOPMENT STAGES AND THE ENGINEERING QUESTIONS THEY EXPOSED.

Stage	Capability developed	Question exposed by use
Integrated prototype	Monitoring-triggered and interactive device investigation	Can the workflow recover from errors and preserve the operator's intent?
Broader operational execution	More platforms, devices, and concurrent work	Does each result mean what the investigation assumes it means?
Evidence and investigation hardening	Better retention, source attribution, and follow-up support	Do later claims remain justified by the available observations?
Reusable validation	Platform fixtures, replay paths, and differentiated review	What behavior has actually been demonstrated on a particular version?

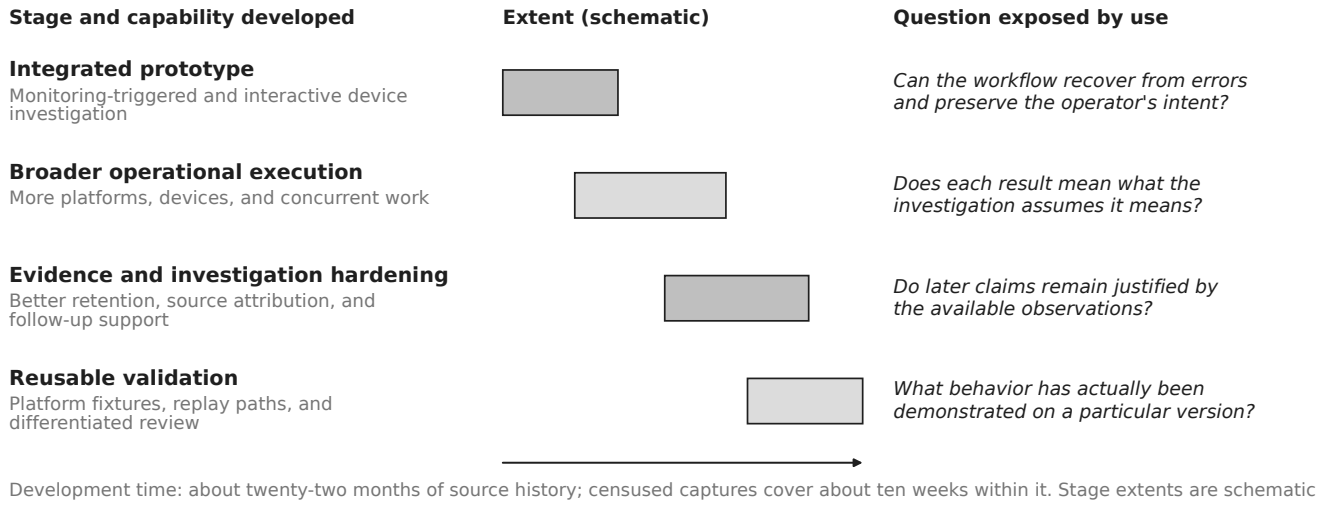


Fig. 4. Development stages recovered from the source history, each with the capability it developed and the engineering question that use of that capability exposed. Stage extents are schematic and overlap; they are not release boundaries.

source, target, collection time, scope, response validity, and retained payload. This is a measurement model for this paper, not the implementation's proprietary evidence format.

For observation e and requirement k, $V(e)$ denotes a usable response, $B(e,k)$ an established association with the target of k, and $S(e,k)$ that the observation's content, scope, and time support k. The binary support indicator is

$$g_{ik} = \mathbf{1}\big[\exists\, e \in E_i : V(e) \wedge B(e,k) \wedge S(e,k)\big]. \qquad (1)$$

We distinguish fractional evidence coverage G_i from satisfaction of every declared requirement C_i:

$$G_i = \frac{1}{|K_i|} \sum_{k \in K_i} g_{ik}, \qquad C_i = \prod_{k \in K_i} g_{ik}. \qquad (2)$$

These quantities depend on the stated K_i. Requirements are stated at the granularity of single observations; a claim that needs several observations jointly, such as a change between two collection times, is expressed as several requirements. Because G_i depends on how finely K_i is divided, it is comparable across investigations only under a common requirement-construction rule, and it is not used here as a cross-task score. A zero support indicator means that support was not established from inspected evidence, not that the requirement is false. Because the existential test does not resolve contradictory observations, $C_i = 1$ is a coverage indicator, not a truth or consistency certificate. For the MTU detection result in Section VI, two required observations are the corresponding endpoint MTUs in the evaluated state. Both-end coverage does not establish traffic causality, because K_i for that larger claim would additionally require path-use and dataplane-impact evidence. Conversely, a rejected endpoint lookup cannot establish a target binding merely because a later command returns real information about some interface (Section X-A).

$V(e)$ is claim-independent: it records whether a response was usable at all, that is, not rejected, incomplete, or unavailable. What an observation supports is claim-relative through $S(e,k)$. A query issued over a successful connection can return three distinct evidence states (Figure 5a). A valid empty table, of which a retained fixture with a table header and no result row is the example in this archive, is usable evidence about the scoped query result: it supports a statement that no matching entry existed within that query's scope at collection time, but not a positive endpoint location, and not that an endpoint never existed, is absent from every relevant device, or must belong to some other interface. An explicit device rejection is evidence about request failure, not a measurement of the requested network property, and it says nothing about whether a matching entry was present. Here K_i concerns the requested

network properties; request-failure classification is analyzed separately. Operational facts are further qualified by source, scope, and collection time. A protocol event before a restart and a healthy interface after it can both be true; a device-local timestamp is not automatically comparable with one from a different clock; a topology observed after a state change cannot be assumed identical to one collected earlier. [E2, E3, E9]

Platform support follows the same logic at four separable levels (Figure 5b): a successful connection, an accepted command, a correctly interpreted result, and a completed diagnostic task. Reporting support at the first level as though it established the last overstates the result.

B. Why a lossy success signal cannot certify the original observation

Let T denote a retained execution trace, $\varphi(T)$ the representation supplied to a downstream consumer, and $h(T)$ a property the consumer is asked to determine. For example, φ may retain a completion label while h asks whether the device rejected the command. The following elementary identifiability condition is useful:

$$\varphi(T_1) = \varphi(T_2),\ h(T_1) \neq h(T_2) \implies \nexists\, g : h = g \circ \varphi. \quad (3)$$

If a decoder g existed, equal representations would imply equal decoded values, $g(\varphi(T_1)) = g(\varphi(T_2))$, contradicting $h(T_1) \neq h(T_2)$. No decision based solely on that representation can be correct for both traces. The statement concerns exact identification from the specified representation; it does not rule out probabilistic prediction or correction using additional evidence.

The condition is not merely a concern about large summaries. A single success label is already a projection. In the retained experiments, normal-return labels coexist with both accepted and rejected responses, so the label alone cannot identify command acceptance. This demonstrates a collision for the label-only projection, not equality of the complete inputs supplied to the historical grader. A consumer restricted to that label cannot validate the omitted distinction from it; a consumer with additional evidence may. [E2, E4]

C. Experimental units and statistical claims

We report finite-cohort frequencies and descriptive differences. The unit of measurement is not necessarily the unit of replication: sixteen command pairs share two executions; ten prompt variants share one induced configuration; repeated baseline prompts share evolving environments. For these cohorts, a binomial interval treating commands or prompts as independent fault deployments would answer an unsupported sampling question.

No hypothesis-test p-value or population confidence interval is used to turn repeated observations into independent replications. Instead, each result states the actual denominator, the source of ground truth, the required evidence, and the variables that changed. This permits exact statements such as "all fourteen selected rejection returns had success labels" while excluding a claim about the rate of all possible diagnostic failures.

V. Platform-Probe Experiment: Successful Transport Is Not Accepted Evidence

A. Protocol and units of observation

We evaluated a read-only command catalog on four devices, $P = \{P_1, \ldots, P_4\}$, using 65 query templates, $Q = \{Q_1, \ldots, Q_{65}\}$. Each device received 56 global templates and nine templates requiring a discovered argument. Commands were issued sequentially through a direct probe harness, bypassing agent planning and answer generation. The intended population was the Cartesian product $P \times Q$: 260 device–template cells. The unit was a particular request on a particular device, not a natural-language investigation. [E2]

We reconstructed outcomes from the original structured execution record and retained device responses. Four cells could not produce a required interface argument and contain no execution result. Therefore $N_{\text{planned}} = 260$, $N_{\text{executed}} = 256$, and $N_{\text{unexecuted}} = 4$. This distinction prevents an argument-discovery failure from becoming either an undocumented omission or an apparent device rejection.

B. Response reconciliation

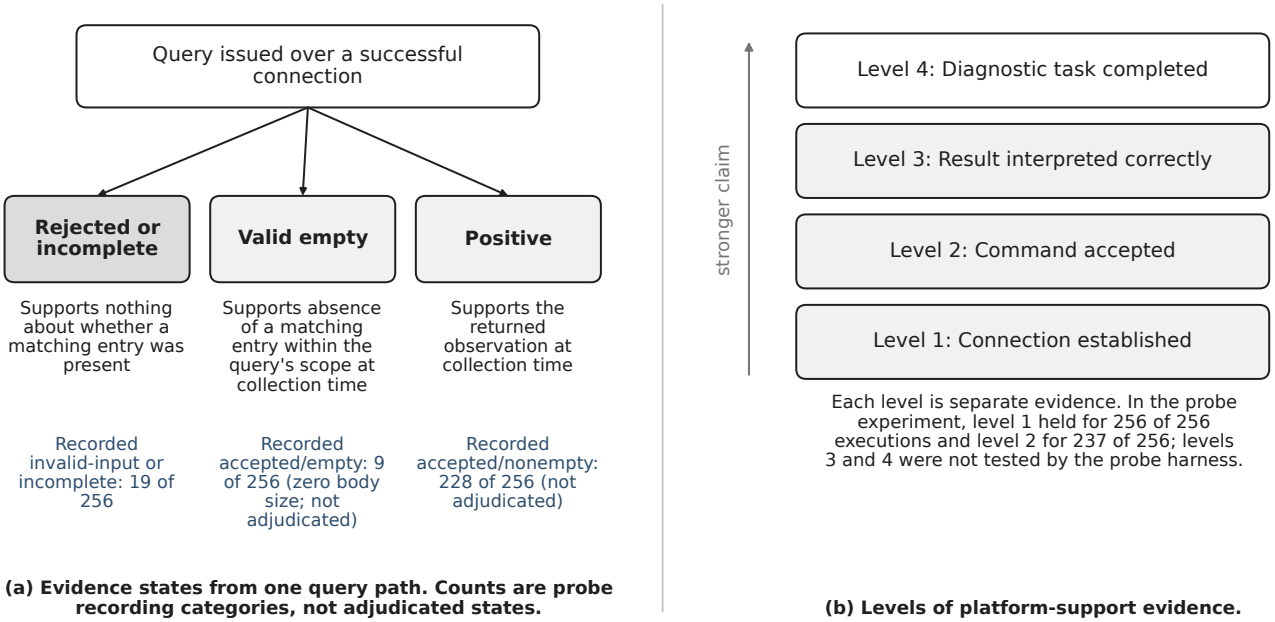
Table IV separates the original harness classifications from explicit device rejection text. "Accepted/nonempty" means the historical probe classified the request as accepted and retained a nonempty body; it does not imply that the requested hardware feature exists or that an investigative question has been answered.

All 256 executed records report normal transport return, including all 19 rejected or incomplete requests. The transport-completion frequency is 100%, while the fraction classified as accepted is $(228 + 9)/256 = 92.58\%$. This 7.42-percentage-point difference is observable without grading a generated answer. Normal return cannot serve as the acceptance criterion. [E2]

All 247 nonempty responses are retained as original files: 228 accepted/nonempty bodies plus 19 rejection bodies. The nine empty cases retain normal-return status, zero recorded body size, and empty inline response fields. The four unexecuted cases instead record missing arguments. These are distinguishable recording states, not interchangeable forms of "no result." They are recording facts rather than adjudicated evidence states: an accepted/empty record does not by itself establish that the response was completely captured and legitimately empty, and an accepted/nonempty record does not by itself establish support for the property the investigation asked about. Figure 5a therefore reports the probe counts as recording categories against the evidence states they approximate.

C. Engineering result

The experiment localized concrete catalog defects (Figure 6). Two proposed query forms were rejected on all four devices, accounting for eight of the 15 invalid-input responses; the remaining seven came from one template rejected on three devices, one on two, and two on a single device each. One template returned an incomplete-command message on all four devices, which establishes an insufficient request, not that



(a) Evidence states from one query path. Counts are probe recording categories, not adjudicated states.

(b) Levels of platform-support evidence.

Fig. 5. Evidence states and evidence levels. (a) A query issued over a successful connection can return a rejection or incomplete-command response, a valid empty response, or a positive observation; each supports a different statement. The counts are the recording categories of the probe experiment in Section V, which approximate the states without adjudicating them: an accepted/empty record is not by itself a validated empty result, and an accepted/nonempty record is not by itself support for the requested property. (b) Platform support is a set of demonstrated behaviors at separate levels; the probe harness tested the first two.

TABLE IV
ORIGINAL PLANNED-CELL AND EXECUTED-RESPONSE RECONCILIATION.

Outcome	Planned cells	Executed responses	Share of executions
Accepted/nonempty	228	228	89.06%
Explicit invalid-input response	15	15	5.86%
Incomplete-command response	4	4	1.56%
Accepted/empty	9	9	3.52%
Required argument unavailable	4	0	
Total	260	256	100.00%

the underlying device feature is unsupported. Two templates produced accepted/empty records on all four devices and a third on one device, and the four unexecuted cells are two argument-required templates on two devices. Accepted syntax also did not guarantee stable output semantics: one query returned a short configuration summary of 139 bytes and four lines on three devices but an event log of 220,408 bytes and 4,593 lines on the fourth. [E2]

A later corrected matrix changes the four missing-argument cells to accepted/nonempty, yielding 232/19/9 accepted-nonempty, rejected-or-incomplete, and empty outcomes. The corresponding correction captures are absent from the inspected archive, so Table IV preserves the initial, executable-record population, and we do not combine the revised labels with original execution counts. The probe establishes an empirical compatibility map and specific failure boundaries; its repeated templates and related devices do not support an independent-trial estimate of diagnostic accuracy.

D. A catalog is not the complete user workflow

Direct syntax onboarding was useful because it revealed actual device behavior. But a probe that bypasses the assistant establishes the behavior of a command path. It does not establish the reliability of natural-language request interpretation, argument generation, dependent collection, or final synthesis, and a command family can be covered while a user-supplied representation or a filtered form remains untested. The field record shows this boundary: an endpoint lookup failed under one input representation and succeeded in a later investigation that used an accepted representation, supplying a real association the earlier investigation lacked. That supports a narrow conclusion about the observed input and response behavior; because software and network state changed during the surrounding evaluation, it is not a randomized comparison, and it does not prove that a deployed automatic correction resolved every downstream problem. [E2, E8]

Laboratory environments supported repeatable protocol and command testing, while physical equipment exposed additional

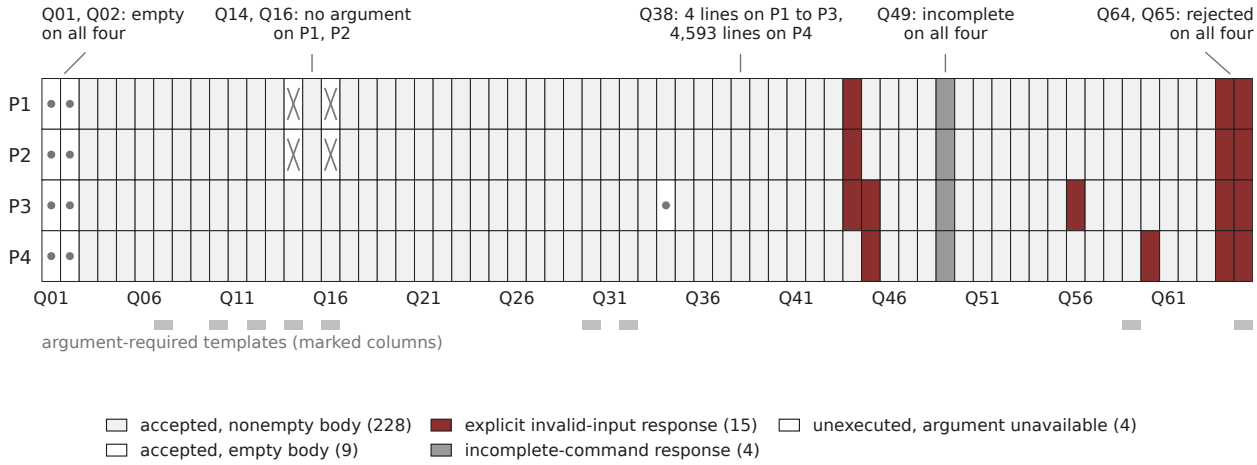


Fig. 6. Outcome of every planned probe cell, four devices by 65 query templates. Cell fills follow the recording categories of Table IV; marked columns are the nine templates that require a discovered argument. All 256 executed cells reported normal transport return, including the 19 rejected or incomplete cells. Annotations name the templates discussed in the text; the flags mark a template across its four planned cells, not additional trials.

operational behavior. Some observations cannot be produced faithfully by a virtual device, and software-image differences change what evidence is available. Replay preserves a captured response and supports regression analysis, but it does not reconstruct every timing, state, or physical condition of the original environment. [E2, E13]

VI. Configuration-State Detection Under Symptom Variation

A. Experimental protocol and observation units

We retrospectively examined a laboratory campaign with two routers connected by two parallel links (Figure 7). The intervention changed one endpoint's MTU on one link from 1500 to 9000 bytes, leaving the opposite endpoint and the parallel link unchanged. A state log records baseline inspection, successful change, verification, ten read-only investigations, restoration to 1500 bytes, and verification of restoration. Transcripts independently preserve both-end interface observations during each trial. Application recovery was not measured. [E3]

The comparison comprises eight pre-intervention investigations from the campaign's router-pair connectivity symptom group and all ten investigations in the modified-state phase. The eight were selected by the original scenario grouping, not by their answers. Their prompts concern connectivity, unreliable traffic, large transfers, intermittency, application instability, abnormal traffic, packet loss, and degradation. The ten modified-state prompts similarly describe symptoms without naming MTU or the injected value. They are distinct prompt variants, not ten independent fault injections; wording is not exactly paired across phases. Investigations ran two at a time with a 480-second per-run timeout. Synthetic symptom descriptions were not accompanied by independently measured application failures. [E3]

The investigation is the reporting unit; the configuration intervention is the replication unit. Eighteen investigations therefore represent only one intervention. The contrast tests configuration-state discrimination under symptom variation, not a software-version ablation or a randomized fault-detection benchmark. Background counters and historical events remained present in the baseline, which does not establish a globally fault-free network.

B. Measurement and adjudication

For each investigation we reviewed the complete delivered answer, recalled the corresponding interface returns, and inspected the retained execution list. Let D_i indicate an explicit identification of the mismatch on the modified link, B_i the presence of both-end supporting observations, P_i a proposed MTU or PMTUD verification, and X_i an executed verification with a returned result. Merely mentioning MTU does not satisfy D_i. A proposed command does not satisfy X_i. A generic counter check without a packet-size or PMTUD test does not satisfy P_i.

In the retained parsed interface records, Endpoint A's interface on the modified link carried a generic `mtu` field with the value 1514 in all 18 captures, alongside an Ethernet encapsulation field; Endpoint B's carried the same field with 1500 before intervention and 9000 afterward. The retained investigations interpret the 1514 value as a 1500-byte IP MTU, consistent with accounting for a 14-byte Ethernet header. The field name itself does not state whether the quantity is a frame or an IP MTU, and the native command-line label was not verified from the retained parsed records; the interpretation is contextual, and it is context-specific normalization rather than a universal instruction to subtract fourteen from every reported MTU. Both endpoints reported their relevant interfaces operationally up during all ten modified-state trials. The comparison is between corresponding interface quantities, not the unqualified numerical inequality of 1514 and 1500. [E3]

We separately reviewed unsupported causal premises. Table V uses Y for a clearly unsupported premise or outcome assertion, A for ambiguous or inconsistent self-assessment wording, and N when no such issue was identified under this narrow rubric. These labels are retrospective and model-

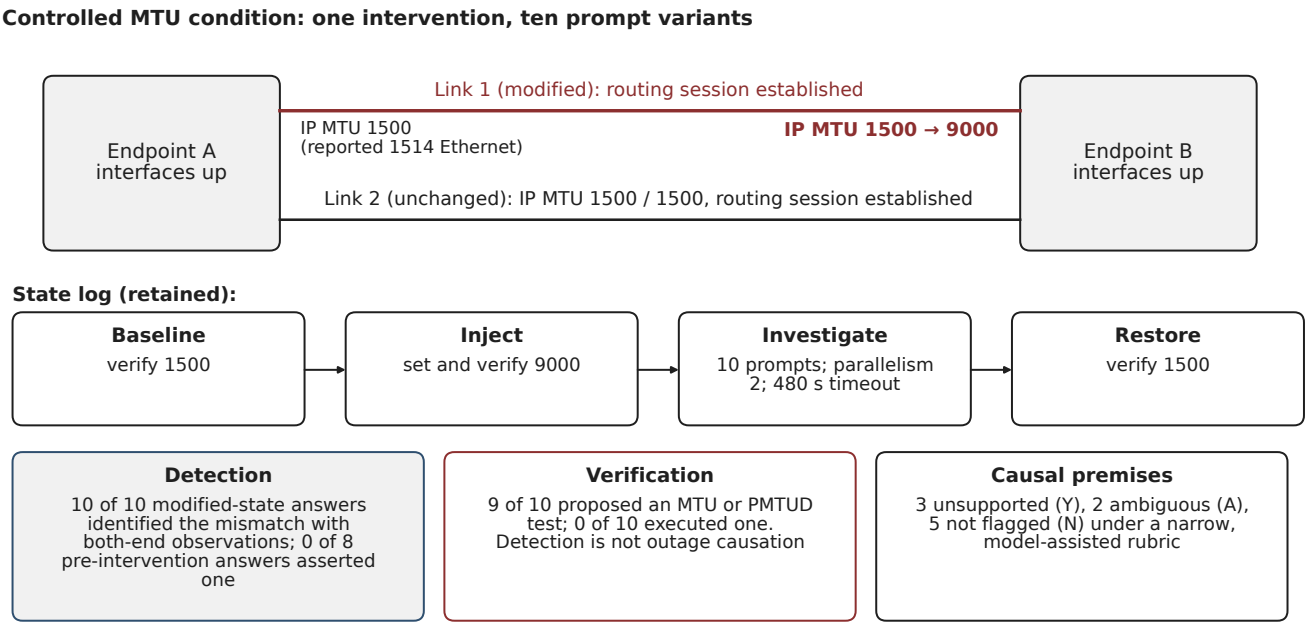


Fig. 7. Logical reconstruction of the controlled MTU experiment. Only one endpoint of one link changes; the parallel link is unchanged, and routing sessions stay established throughout. MTU values are the retained experimental settings; endpoint identities and operational addresses are omitted. The lower row summarizes the adjudicated outcomes of Section VI-C.

authored (Section II-F); they are not whole-answer correctness scores.

C. Results

All ten modified-state answers explicitly identified the mismatch and had both-end support: $\sum_i D_i = \sum_i B_i = 10$. All ten retained a final answer and a zero process exit. Their execution lists contain 93 device queries. Nine answers proposed MTU or PMTUD verification, but none executed it: $\sum_i P_i = 9$, $\sum_i X_i = 0$. The remaining answer proposed forwarding-path and counter checks instead. None of the eight reviewed pre-intervention answers asserted an MTU mismatch; retained interface returns in all eight support the matched, normalized MTU state. [E3]

The mean observed interval was 131.5 seconds (131.498 at retained precision), the median 124.8 seconds (124.844), and the range 105.6 to 178.4 seconds. These descriptive intervals include reasoning and other recorded processing under two-investigation concurrency; they establish neither human time savings nor a deployment-wide latency distribution.

Correct configuration detection did not ensure correct causal reasoning. M04 assumed that equal-cost multipath forwarding was in use without collecting routing or forwarding evidence. M08 asserted coincidence with a degradation window that was neither supplied nor measured. M10 inferred unaffected standard-size dataplane traffic from established routing-protocol sessions. M01 and M03 retained inconsistent self-assessment references to a confirmed cause although their delivered assessments used hypothesis language and acknowledged missing packet-level verification. The remaining five were not flagged under this rubric, but they were not certified correct in every respect. [E3]

The demonstrated result is detection of a specific configuration anomaly that a simple link-up or session-up check would not reveal, including correct treatment of heterogeneous MTU representations. Traffic impact, flow selection, and application causality remain unverified. The repeated investigations share devices, configuration state, and execution conditions, so ten detections are not ten independent fault-recovery successes (Section IV-C).

VII. Rejection Semantics, Evaluator Disagreement, and Staged Correction

A. Cohort and event definition

We examined all sixty retained transcripts and corresponding automated grade records from a separate overnight campaign. The transcripts contain 510 delimited tool-output previews. Candidate rejections were located using declared invalid-input, unknown, incomplete, or ambiguous-command, syntax-error, and unrecognized-command expressions; each selected event was then checked against its original tool-return boundary and invocation identity. This search covers those expressions, not every possible execution failure. [E4]

The review identified fourteen distinct invalid-input rejections across seven investigations, all involving the same uptime query against two laboratory devices. Two narrative repetitions were excluded. Each of the fourteen actual rejection returns was accompanied by a completion label saying “Success.” The distinction is therefore between transport or execution completion and device acceptance, not between fourteen different failure mechanisms. The enumeration concerns the retained visible previews, not an exhaustive scan of every full device response; the fourteen selected rejection bodies are short

TABLE V
MODIFIED-STATE INVESTIGATIONS. ALL ROWS DETECTED THE MISMATCH, RETAINED BOTH-END SUPPORT, AND PRODUCED AN ANSWER; NONE EXECUTED AN ACTIVE MTU TEST. TIME IS THE INTERVAL BETWEEN THE EARLIEST AND LATEST RETAINED INVESTIGATION LOG TIMESTAMPS, NOT DEVICE EXECUTION TIME ALONE. Y/A/N IN THE FINAL COLUMN CONCERNS CAUSAL PREMISES ONLY.

Case	Symptom family	Device queries	Time (s)	MTU test proposed	Causal review
M01	Lost connectivity	9	110.7	Yes	A
M02	Unreliable traffic	13	178.4	Yes	N
M03	Large-transfer failure	3	105.6	Yes	A
M04	Intermittency	10	122.0	Yes	Y
M05	Application instability	9	114.2	No	N
M06	Abnormal traffic	9	120.9	Yes	N
M07	Flaky reachability	8	127.7	Yes	N
M08	End-to-end degradation	10	136.3	Yes	Y
M09	Packet/session loss	12	148.9	Yes	N
M10	Most-likely-problem request	10	150.4	Yes	Y

and fully visible, and other failures outside retained previews are not ruled out.

B. Disagreement with the historical evaluator

Every affected investigation had a historical invalid-command count of zero. Five received the evaluator's top overall category and two were marginal; all seven received its highest platform-correctness score (Figure 8a). Table VI preserves the disagreement without treating the historical categories as expert ground truth. [E4]

The mismatch affected 7 of 60 investigations and 14 of 510 retained tool-output previews. These are descriptive incidences for the recognized rejection family, not whole-answer error rates. The evaluator did register three invalid-command events in two other investigations, so the result is not that it reported zero errors everywhere.

Inspection of the retained evaluation procedure explains the blind spot. Its invalid-command metric examined completion labels, while its judgment input contained a bounded command list, brief reasoning, and an answer excerpt rather than the full device-return evidence. A rejection concealed by the completion label could therefore remain absent from both the metric and the judge's evidence. A rejected query does not necessarily invalidate the entire investigation, but it directly contradicts a claim that all executed queries were accepted or platform-correct. [E4]

Let R_i indicate that investigation i contains a selected rejection and z_i its historical invalid-command count. The observed conditional miss fraction is

$$f = \frac{\sum_i R_i \cdot \mathbf{1}[z_i = 0]}{\sum_i R_i} = \frac{7}{7}. \tag{4}$$

This equals one for the seven recognized affected investigations. It is not an estimated false-negative rate over unseen failure families or a whole-answer error rate.

A broader consequence follows for model-assisted evaluation. An evaluator that is given neither the underlying observation nor an independently grounded equivalent cannot independently verify that observation, which is the practical form of the identifiability condition in Section IV-B. Asking a model to adopt an expert role does not make its judgment an independent expert review, and a model's confidence or critique is not a measured calibration result. These distinctions have to be stated when interpreting the historical grades.

C. Four observable correction stages

The retained follow-up artifacts support a staged engineering result (Figure 8b).

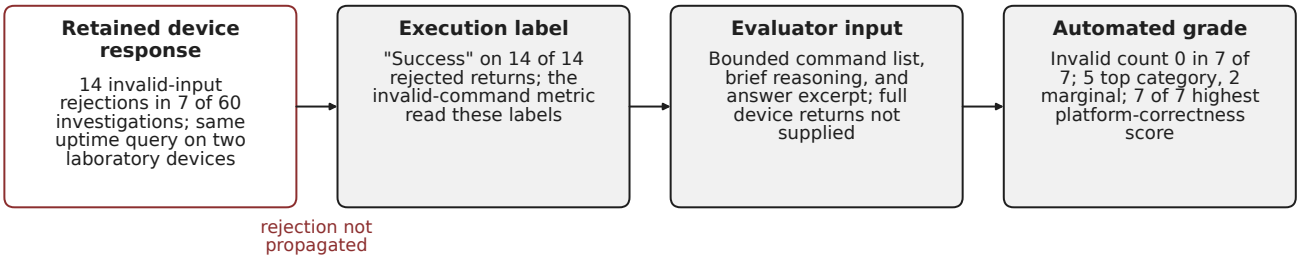
1) *Original behavior.* Fourteen bare rejection returns had misleading success labels.
2) *Payload correction.* Seven matched-prompt reruns contained 142 delimited tool-output previews. All fourteen corresponding rejections now carried explicit failure status, but all fourteen completion labels still said success. Correcting the evidence representation had not corrected every consumer.
3) *Presentation correction.* A separately retained targeted invocation showed both an explicit rejection payload and an error completion label. This validates the corrected presentation path for one selected invocation, not a repeated seven-case benchmark.
4) *Subsequent legitimate success.* Another targeted invocation returned actual uptime data, a success label, and a corresponding final summary. Separately retained lower-layer and wrapper results also show successful uptime output. This establishes that the query was not inherently unsupported; these artifacts alone are not a randomized access-mode comparison. [E4]

The seven reruns preserved the affected prompts, but other execution conditions were not experimentally frozen. The defensible improvement is therefore narrower than a global reliability gain: explicit rejection representation was observed in fourteen of fourteen corresponding returns after correction, and a later targeted check established propagation to the presentation boundary. Evidence representation, consumer behavior, and evaluator inputs require separate validation. A correct intermediate status is insufficient when another consumer still promotes it as success.

TABLE VI
INVESTIGATIONS CONTAINING RETAINED DEVICE REJECTIONS. EACH INCLUDES TWO REJECTED RETURNS LABELED SUCCESSFUL. THE LAST TWO COLUMNS DESCRIBE MATCHED-PROMPT RERUNS AFTER THE FIRST CORRECTION.

Case	Original rejections	Grader invalid count	Historical category	Rerun explicit rejections	Rerun success labels
E01	2	0	Top	2	2
E02	2	0	Top	2	2
E03	2	0	Marginal	2	2
E04	2	0	Top	2	2
E05	2	0	Marginal	2	2
E06	2	0	Top	2	2
E07	2	0	Top	2	2

(a) Rejection returns, execution labels, and evaluator inputs in the 60-investigation cohort



(b) Four observable correction stages reconstructed from retained captures

Stage	Observed context	Payload	Visible status
A	7 investigations / 14 rejections (original)	Unclassified rejection text	14 success labels
B	7 reruns / 14 rejections (142 previews)	**14 explicit rejection statuses**	**14 success labels**
C	1 targeted rejected query	Explicit rejection	**Error label**
D	1 targeted valid-query check	Valid uptime data returned	Success label

Stage B corrects the payload but not the consumer that renders the label. Stages C and D are single targeted checks, not repeats of the seven-run cohort; D establishes valid output in its tested context only.

Fig. 8. (a) How fourteen device rejections in seven of sixty investigations reached the automated evaluator as successes: the execution label concealed the rejection, the invalid-command metric read that label, and the judge's input omitted the full device return. (b) Four correction stages reconstructed from retained captures. Stage B corrects the payload representation while the visible label is still wrong; Stages C and D are single targeted checks rather than repeats of the seven-run cohort.

VIII. MATCHED-WORKLOAD LATENCY EXPERIMENT

A. Recorded revisions and measurement definitions

Five retained laboratory runs, L1 to L5, document an execution-setup refinement sequence. The common request concerns adjacency discovery and routing-session summaries across eight devices. L1 is an intermediate experimental revision, not the original system baseline. L2 adds an initialization refinement; L3 further reduces repeated per-command setup; L4 reruns the refined path; L5 tests a temporary pacing variant. These describe intervention categories, not proprietary implementation logic. [E5]

We extracted each device–command identity and its displayed duration from persisted result events. Durations are quantized to 0.1 seconds. We use the ordinary sample median: the central value for odd n and the average of the two central ordered values for even n. Inventory queries are counted separately. The request lifecycle L runs from the recorded request-received event to processing completion; it is not the sum of concurrent command durations.

B. Matched command-set result

L1 and L3 contain exactly the same 16 device–command pairs: two commands on each of eight devices. L3 additionally performs one inventory query. For matched device–query pair j, let d_{1j} and d_{3j} denote its displayed durations in L1 and L3. Define the paired difference and the marginal-median reduction fraction by

$$\Delta_j = d_{3j} - d_{1j}, \qquad r_m = 1 - \frac{\mathrm{med}(d_3)}{\mathrm{med}(d_1)}. \tag{5}$$

TABLE VII
FIVE RETAINED EXECUTION VARIANTS. L1 IS AN INTERMEDIATE REVISION; DIFFERING CALL POPULATIONS MUST NOT BE TREATED AS INTERCHANGEABLE WORKLOADS.

Run	Device commands	Inventory queries	Median command duration (s)	Command range (s)	Lifecycle L (s)
L1	16	0	12.15	4.3 to 14.9	81.74
L2	23	1	7.30	4.2 to 9.3	94.23
L3	16	1	4.60	2.3 to 5.1	64.18
L4	16	0	5.05	3.1 to 6.7	61.39
L5	24	1	3.95	1.8 to 8.9	195.12

Fourteen displayed durations decrease; two increase, by 0.1 and 0.2 seconds. The median paired difference is -8.30 seconds, which differs from subtracting marginal medians, $4.60 - 12.15 = -7.55$ seconds (Figure 9a). [E5]

The command median decreases descriptively by $100(1 - 4.60/12.15) = 62.14\%$. The request lifecycle decreases by 21.49%, using unrounded timestamps: 81.741823 to 64.175003 seconds. L4 repeats the same device-command set and completes in 61.389609 seconds. These observations support a substantial improvement in the matched workload while exposing two individual regressions.

C. Locating the remaining latency

Four timestamps define a telescoping decomposition: t_0, request receipt; t_1, first device-collection start; t_2, last worker completion; and t_3, processing completion:

$$L = t_3 - t_0 = (t_1 - t_0) + (t_2 - t_1) + (t_3 - t_2). \quad (6)$$

The middle interval includes parallel collection and worker reasoning. The final interval covers post-worker processing through the recorded completion stamp, including review and synthesis. The stamp precedes the terminal assistant marker in these captures; successful rendering and delivery are not timed by this endpoint. Neither interval is a pure network or pure model timer.

The middle interval decreases 53.74% (38.314 to 17.723 seconds at retained precision), while the post-worker interval remains approximately unchanged and is the largest remaining measured interval in L3 (Table VIII, Figure 9b). The decomposition accounts arithmetically for the smaller request-level percentage reduction; it does not identify the internal component responsible for the remaining time. The same decomposition for the other runs is descriptive only: L2's pre-collection interval is 31.29 seconds against 6.81 in L1, and the records do not attribute that difference to a specific cause; L4 reproduces the shape of L3; and L5's regression is concentrated in the collection and worker interval, 136.50 seconds, consistent with its longest worker. The engineering outcome is both a faster observed processing interval and a measured boundary for further profiling. [E5]

D. Failed trial and inference boundary

L5 has a lower command median than L4, yet its lifecycle reaches 195.12 seconds. It also adds routing-table collection and executes 24 rather than 16 device commands; its longest worker takes 136.281 seconds. The trial missed the run-level objective, but changed scope and worker reasoning prevent attributing the regression solely to pacing. Dividing lifecycle by command count would not remove those confounds. A metric that reported only the median tool duration would have missed the run-level regression entirely.

The evidence consists of five individual recorded runs, including the refined-path rerun, without randomized order or controlled host and model timing. Immutable build identity is not established for every run. Matched commands are clustered within a shared run, so treating the 16 pairs as independent experimental replications would overstate statistical evidence. These results quantify a recorded engineering improvement and its measured interval limits, not a fleet-wide latency guarantee or a randomized causal effect. Some historical human-equivalent time values in the archive were calculated from assumptions rather than measured human task performance; we do not report them as productivity results. [E5, E6]

IX. COMPLETION INTEGRITY AND LONGITUDINAL PROCESSING MEASUREMENTS

A. Repeated-baseline measurements

Table IX reports the ten campaigns in the selected repeated-baseline series, in chronological order (Figure 10a). The same 25 exact prompt strings recur, but the campaigns are not a controlled single-variable ablation. Processing medians range from 62.912 to 145.377 seconds. These are measured engineering observations, not evidence of a specific mechanism's causal speedup. [E6]

The result rows classify 247 observations as process-level successful exits, two as failed exits, and one as a timeout. All 250 transcripts contain completion stamps, and all 250 report rows assert that an answer was produced. However, only 248 transcripts contain a delivered-assistant marker. In the two failed-exit cases, a table-rendering exception occurs after the completion stamp and before terminal answer delivery. The archived traceback supplies the failure location; the report's answer-produced flag does not establish successful delivery. The timed-out invocation remains in the table rather than being silently removed. [E6]

Process completion, reported answer production, and successful presentation therefore cannot be treated as interchangeable success criteria. The 247 zero exits are not 247 verified

TABLE VIII
NON-OVERLAPPING EVENT-DEFINED INTERVALS FOR ALL FIVE RETAINED RUNS, FROM RETAINED-PRECISION EVENT TIMESTAMPS. ROUNDED DISPLAY VALUES NEED NOT SUM EXACTLY.

Interval	L1 (s)	L2 (s)	L3 (s)	L4 (s)	L5 (s)
Before collection	6.81	31.29	9.66	10.65	10.90
Parallel collection and worker reasoning	38.31	24.99	17.72	17.65	136.50
After worker completion	36.62	37.94	36.79	33.09	47.72
Total	81.74	94.23	64.18	61.39	195.12

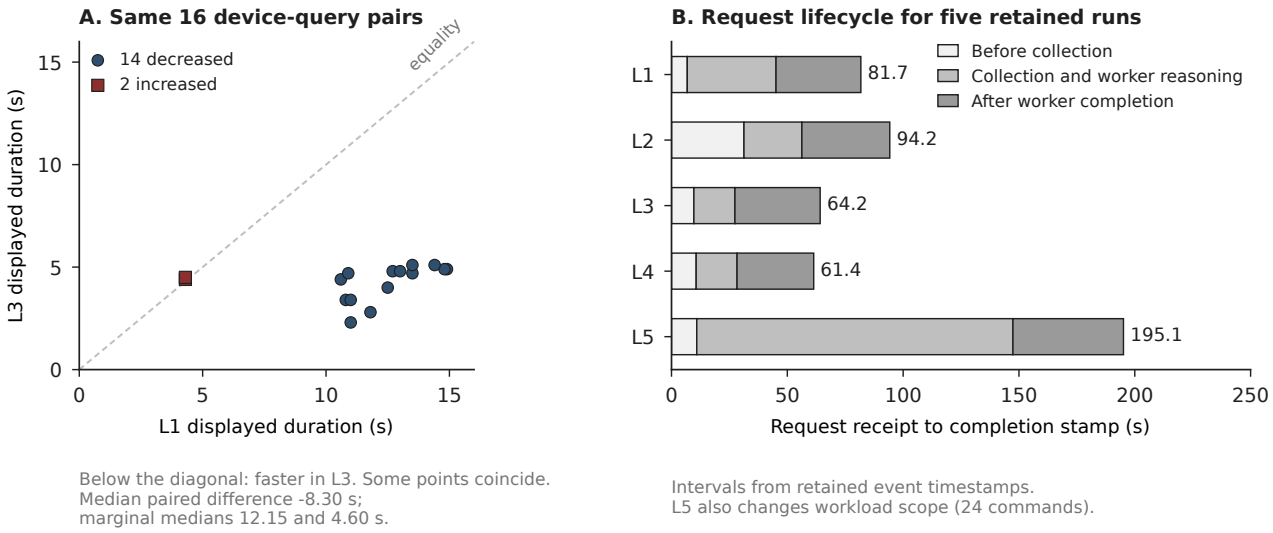


Fig. 9. Matched-command latency and request lifecycle. (a) The same sixteen device–query pairs, plotted as displayed duration in L1 against L3 with the equality diagonal; fourteen pairs lie below it and two lie marginally above. (b) Request receipt to completion stamp for all five runs, with the event-defined decomposition of Table VIII. Command distributions and request intervals are different observables; L5 changes workload scope and is not a controlled pacing ablation.

TABLE IX
TEN CAMPAIGNS IN THE SELECTED REPEATED-BASELINE SERIES. PROCESS EXIT CATEGORIES AND OBSERVED INTERVALS ARE NOT DIAGNOSIS GRADES.

Campaign	n	Zero exit	Failed	Timeout	Median processing (s)	Observed range (s)
B01	25	25	0	0	122.034	14.927 to 171.612
B02	25	25	0	0	138.634	15.096 to 203.026
B03	25	25	0	0	112.542	9.134 to 225.050
B04	25	25	0	0	100.086	7.783 to 137.909
B05	25	25	0	0	117.968	8.987 to 246.554
B06	25	24	0	1	97.657	46.821 to 457.162
B07	25	24	1	0	125.859	38.401 to 262.481
B08	25	25	0	0	145.377	37.043 to 349.782
B09	25	24	1	0	128.470	42.457 to 267.680
B10	25	25	0	0	62.912	40.366 to 95.487

diagnoses, and the 250 answer-produced flags are not 250 successful operator handoffs. [E6]

B. Alert-result consistency

Alert-triggered work imposes obligations beyond interpreting the text of an alert: the system has to associate events correctly, preserve relevant context across background execution, and expose what happened when processing was delayed or interrupted. The alert harness supplies a separate collection of completion and answer-presence measurements against those obligations. For result i, let c_i be one when the initial-turn-completed flag is true and the failed flag is false; δ_i the number of completions recorded for the current invocation slice; and ℓ_i the current final-answer length. Define the concordant stored-

field indicator

$$A_i = \mathbf{1}[c_i = 1 \wedge \delta_i > 0 \wedge \ell_i > 0], \qquad \sum_i A_i = 110. \quad (7)$$

Across 118 dispatched results, 116 satisfy $c_i = 1$, 113 additionally satisfy $\delta_i > 0$, and 110 also satisfy $\ell_i > 0$ (Figure 10b). These are nested checks on stored fields, not independently adjudicated diagnostic outcomes. [E6, E12]

All relevant keys are present. Three records satisfy $c_i = 1$ but have no new messages, no current-slice completion, and no current final answer. This is disagreement between completion flags and run-relative evidence; the historical cause is not established here. Three different records have $c_i = 1$ and $\delta_i > 0$ but lack a nonempty final answer. Targeted transcript inspection shows planning or thinking followed by completion without a new substantive final response. Earlier answers visible in a reused conversation are not credited to the new alert. [E6]

The two explicitly failed records report that the initial turn did not complete. All 118 records report clean final state, demonstrating that queue or lock cleanliness alone is insufficient to establish useful completion. Sixteen records contain a follow-up block; all report completion without an error, but 15 record zero follow-up elapsed time, and those zero fields are excluded from latency interpretation. [E6]

The 110 records satisfying all three predicates establish agreement among stored completion and answer-presence fields, not 110 correct investigations. Fixtures include repeated alerts and unavailable targets; assessing correctness additionally requires the expected behavior and the content of the response. The harness runbook also excludes browser and socket end-to-end testing, so a fixture replay exercises only the path and assumptions represented by that fixture. Pasting an alert into a chat demonstrates neither receipt through an external provider's webhook nor the full interval from alert generation to an actionable response, and earlier generic fixtures are not proof of later customer-side completion. [E6, E12]

C. Additional audit-recorded terminal-outcome failures

The retained archive contains further cases, recorded by earlier internal reviews, at the same boundary. An earlier worker log records a model context-limit error followed by nominal task completion with no returned answer. A later stopped investigation was recorded with a successful completion label. Another laboratory review distinguished a reasonable model output from the result actually delivered. These observations occurred at different boundaries and should not be collapsed into one defect. Present source inspection does not prove that every started investigation receives exactly one correct terminal disposition; durable outcomes are an engineering requirement and an implemented area of work, not an experimentally established universal property. [E8, E12]

X. Field Evaluation: Target Binding, Evidence Handling, and Read-Only Scope

The bounded external evaluation differs from the laboratory campaigns in three ways: fault ground truth was generally unavailable, software and network state changed between requests, and the findings are audit-recorded from the team's forensic record rather than machine-censused. The cases below are therefore reported qualitatively, with the same distinction between observation and verified relationship that Section IV-A formalizes.

A. Finding an anomaly is not locating the user's problem

The field evaluation produced useful observations about interface instability, protocol events, and device state. In some investigations, however, the system found an abnormal interface without establishing that the requested endpoint was attached to it. The observation was real; the relationship needed to answer the question was not verified. [E8]

A schematic example makes the difference clear (Figure 11). Suppose evidence establishes that interface P is down, and a lookup for endpoint E fails. The interface observation does not establish that E uses P. The symbols are invented; this is not a reproduction of a customer record. It expresses the inferential error the audit documented: a plausible association was promoted to a verified association without the necessary evidence. The same lookup difficulty did not produce one uniform outcome. The audit records false absence, honest disclosure of a lookup failure, and unsupported attribution in different investigations. The defect therefore cannot be reduced to a command problem; how the system interpreted the failure determined whether the final answer stayed appropriately limited.

A later internal investigation documented a valid empty lookup followed by collection from an interface that had not been linked to the target. The dependent command succeeded and returned real protocol information. The unsupported step was the premise used to select and describe that interface. Final synthesis then recovered by stating that the relationship was not verified. This is a positive result and a remaining weakness in one trajectory: the final qualification kept the unsupported relationship out of the conclusion, but intermediate narration and collection had already proceeded on it, and judging final-answer quality alone would miss the incorrect intermediate claim and the work it caused. Our engineering work therefore treated target identification and evidence attribution as investigation responsibilities rather than assuming successful tool execution would validate them. Current implementation and focused test definitions support the existence of related checks; they do not establish universal correctness of the relationship, its freshness, or the causal diagnosis built on it. [E10]

B. Acting on discovered dependencies and separating state from cause

Some requested evidence depends on information discovered during the investigation. A broad inspection may identify which objects need detailed measurements, so an initially successful command can be the start of a diagnostic sequence rather than its end. Our development records contain cases where the assistant correctly reported missing detail after discovering the relevant objects but did not complete the dependent collection. A wording revision alone did not resolve one such case. Later work investigated and implemented bounded continuation, with

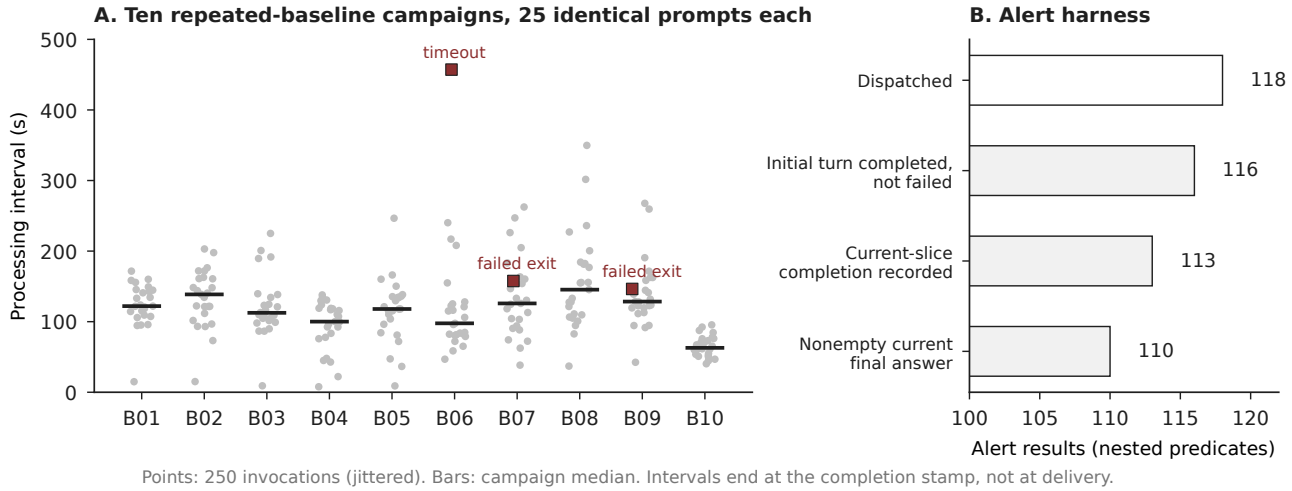


Fig. 10. Completion integrity across campaigns. (a) All 250 recorded processing intervals for the ten repeated-baseline campaigns of Table IX, with campaign medians; the three non-zero exits are marked. (b) Nested stored-field predicates for the 118 alert-harness results: 116 completed initial turns without a failed flag, 113 also recorded a current-slice completion, and 110 also carried a nonempty current final answer.

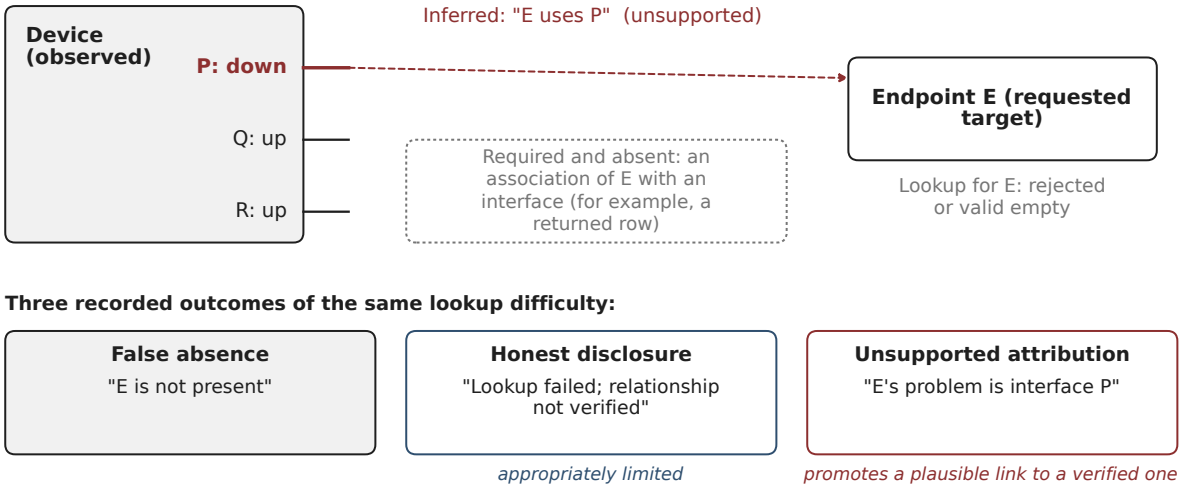


Fig. 11. Schematic of unsupported endpoint attribution in the field record. Interface P is observed down; a lookup for endpoint E is rejected or returns a valid empty result. The association "E uses P" is not supported by either observation, and the evidence that would support it is absent. The same lookup difficulty produced three different recorded outcomes; only honest disclosure kept the final answer appropriately limited. Symbols are invented and do not reproduce a customer record.

reported validation and explicit enablement limits; earlier design conclusions in that record were later superseded. Language-model-led planning needs an execution environment that can support the intended investigation, and the converse also holds: allowing another step does not establish that the step is relevant or justified. Completeness and restraint have to be judged against the question, the available evidence, and the unresolved dependencies, not against the number of commands performed. [E11]

Restart investigations showed a related boundary. Current uplink state, protocol operation, and power-delivery observations can support a limited recovery assessment. They do not necessarily identify the cause of a previous restart, and they do not prove that every expected downstream service recovered. Missing measurements on a relevant device also cannot support a claim that a hardware cause was ruled out. In the same way, degraded link behavior may justify investigating a physical segment without uniquely identifying a defective component. A hypothesis can be useful while alternatives remain; the objective is to keep hypotheses distinct from observations and from verified causal conclusions, as the MTU causal review in Section VI-C also required. [E8]

C. Evidence handling across summarized context

Operational commands can return far more material than a given question needs. Retaining that material is valuable for inspection, but presenting all of it to every reasoning step is not necessarily useful, and reducing it to a summary can remove the exact fields needed to answer the operator's question. One internal rehearsal recorded successful collection of a protocol-status view across several devices, followed by unequal preservation of global and per-interface fields in the material available for synthesis. Another recorded uncertainty about retained-log boundaries and startup evidence because the final answer relied on summaries. These are audit-recorded observations; the full historical traces were not retrieved again during preparation. They identify a distinct failure boundary: collection can succeed while the evidence available for the next decision becomes incomplete. We built support for retaining collected evidence and for revisiting the relevant source material when compact representations were not enough, and later internal records document bounded validations of exact-detail recovery. Neither observation establishes indefinite retention, complete recovery under all conditions, or semantic correctness simply because retrieval occurred. [E9, E10]

A summary can reasonably convey that instability was observed. An exhaustive event list, an exact count, or a precise chronology needs a stronger evidentiary basis. If the visible material is incomplete, the agent must establish the relevant source coverage or narrow the claim, and omission from a summary is not evidence of absence from the underlying record. The same rule applies to searching sources: matching lines are not necessarily distinct events, one event can appear in raw output, in a progress message, and in a final answer, and repeated representations of the same evidence do not provide independent corroboration. Rehearsal notes also identified uncertain clock labeling, and the field audit recorded real environment changes between repeated requests; these findings support preserving uncertainty about timing and state, rather than treating every difference between answers as hallucination or every consistent answer as repeatability. [E8, E9]

A stored result can be irrelevant to the current claim, insufficiently scoped, or no longer available when a later audit begins. A source reference establishes where evidence came from; it does not establish what that evidence proves. The practical requirement we derived is that a claim must remain traceable to an appropriate observation and to the limits of that observation, and that requirement applies to the assistant's answer and, equally, to retrospective reports about the assistant.

D. Read-only scope and governance

The program included device scoping, command validation, authorization for configuration actions, and tests of permitted execution boundaries. These capabilities matter because a troubleshooting agent operates against systems whose state matters. They are not equivalent to complete enterprise identity management, approved data processing, or security certification. The field cases emphasized read-only investigation, which limits what they can establish about remediation: identifying a likely fault and suggesting an action do not demonstrate safe autonomous repair, verification after change, or rollback, and this paper makes no such claim. Where configuration workflows existed in the broader product, their implementation is kept separate from the read-only diagnostic results discussed here. [E12]

E. Positive field results within their limits

The internal field audit records useful event correlation, a later endpoint association supported by a returned row, and collection of inventory and reachability information. It also records repeated investigations that strengthened corroboration of an apparent event origin. These results demonstrate bounded investigative capability as documented by the team's audits. Their significance does not depend on claiming that every fault was independently confirmed or that every response satisfied the full task; a useful partial finding and an incomplete investigation can coexist, and preserving both is more informative than reducing the program to either a set of successful demonstrations or a list of defects. [E2, E8]

XI. Evaluation Practice and Validation Requirements

A. Multiple instruments with different purposes

The program developed and used command probes, baseline investigations, planted-condition laboratory tests, retained-output checks, automated grading, human-assisted review, and field forensics. It would be wrong to describe the history as unevaluated, or to treat one unattended campaign as the whole evaluation population. The difficulty was comparability and coverage. A command probe measures whether a particular interface behaves as expected; a planted-condition test can assess whether the platform detects that condition; a human-assisted review can examine the delivered answer; an automated judge may assess only a limited representation of the trajectory. These instruments give useful but non-equivalent evidence. The MTU state log that sets, verifies, and restores a planted condition, and the baseline review that separates clean process exits from a failed delivered answer, are meaningful controls; they do not produce an independently adjudicated global accuracy rate. [E3, E6]

B. Reports require verification too

The manuscript review found discrepancies in reported command-probe totals and in summaries of historical execution outcomes, and it found status headers contradicted by later entries in the same document. Narrative counts of one error condition did not reconcile cleanly with the retained transcripts and evaluation records, and those derived totals were excluded from the reported results. These are evidence-management findings, not grounds to discard the engineering record. The right response is to identify the relevant source version, check the underlying rows where available, and narrow the claim if the records do not reconcile. A later correction is not exact authority simply because it corrects an earlier report. This paper follows that rule by reporting only reconciled counts and by preserving the distinction between a directly inspected artifact and a prior audit's account of it. [E2, E4]

C. A passing run may not test the intended change

One informative internal addendum retracts an earlier interpretation of a live validation. The run was useful as a check for regressions, but it did not exercise the specific behavior attributed to the change. Evidence for that behavior came from separate synthetic and captured-condition tests. A revision can be present without its relevant path being exercised; a component can pass without the whole investigation succeeding; a deployment can complete before its intended device validation. The staged correction record in Section VII-C is the counted instance of the same principle. The work is real at each stage, but the stage determines what can be claimed. [E10]

D. Validation must follow the operator's complete question

Table X states the evaluation requirements derived from the observed gaps. Each row names the failure mode that one of the experiments or audits in this paper exhibited. They are proposed requirements for future controlled evaluations, not a claim that this paper has completed every test listed.

XII. Synthesis

A. RQ1: success labels fail at distinct, measurable boundaries

The probe experiment gives an explicit counterexample to normal transport return implying command acceptance: 19 of 256 normally returned requests contain rejection or incomplete-command responses. The evaluator study shows the same distinction surviving into an apparently favorable assessment: fourteen selected rejection spans have successful labels, and five affected investigations receive the top historical grade. The completion study exposes a later boundary: two failed baseline invocations have completion stamps but no delivered-assistant marker, while six nonfailed alert records lack one or more current-invocation completion or answer indicators. The field record adds a boundary the laboratory did not reach: a real anomaly presented as the requested endpoint's problem without a verified association. [E2, E4, E6, E8]

These are not interchangeable instances of an undefined "hallucination" rate. They concern request acceptance, target attribution, evaluator visibility, attribution to the current invocation, and terminal presentation. Their denominators and consequences differ. The useful result is localization: an engineer can specify the failed implication and validate the responsible boundary without treating a favorable overall outcome as proof that every intermediate property holds.

B. RQ2: intervention validation must identify the corrected behavior

The rejection sequence demonstrates a real but staged improvement. The first rerun cohort preserves explicit rejection status in fourteen corresponding returns while the visible completion labels remain wrong; a separate targeted run then demonstrates corrected failure presentation, and another demonstrates legitimate successful output. Collapsing these stages would lose the evidence needed to distinguish a representation fix from a consumer fix. [E4]

The MTU campaign establishes a different engineering capability. Under ten symptom descriptions that do not name the injected condition, the system identifies the actual mismatch using both endpoints' observations, and the eight pre-intervention cases provide a matched-configuration contrast rather than a claim of universal network health. The unexecuted dataplane checks and the causal-premise concerns define its boundary. The field cases show the complementary failure: correct data collected and then attached to an unverified target. [E3, E8]

These results support retaining separate evaluation labels for observed anomaly, required-evidence coverage, justified relationship, causal attribution, and operator delivery. Such labels are analytical outcomes to adjudicate, not a proposal to replace the agent's reasoning with a deterministic diagnostic rule engine.

C. RQ3: the application interval limits observable speedup

The matched-command comparison improves the displayed command median by 62.14% but the request-to-completion-stamp interval by 21.49%. The event decomposition accounts for this difference in the retained runs: the collection and worker interval contracts, while the post-worker interval remains near 36.7 seconds. This is an arithmetic decomposition of elapsed time, not identification of an internal causal bottleneck or a claim about unobserved CPU, network, or model-service time. The pacing trial reinforces the need to measure the entire request: its lower command median coexists with a much longer lifecycle and a changed workload. [E5]

D. What the engineering program established

The program produced an implemented agentic NetOps platform for troubleshooting, with multiple platform integrations, interactive and alert-driven execution, control over permitted actions, and facilities for retaining and reviewing investigative evidence. Their existence is supported by source history and implementation records; their operational reach is qualified by the bounded validations reported here. [E7, E12]

The evaluation record demonstrates useful investigations under stated conditions. It also demonstrates that the hardest failures were not always missing facts. Some involved the status of a fact, the relationship connecting it to a target, or the difference between a completed operation and a completed investigation, and they were consequential even when commands returned real data and the assistant showed substantial networking competence. Broad platform access did not eliminate platform-specific testing. More collected data did not guarantee more useful evidence. A stronger explanation did not guarantee adequate collection. A passing evaluator did not establish correctness of the raw interpretation. A successful follow-up result did not establish complete acceptance of the revised system. [E2, E3, E4, E8, E9, E10]

These findings do not imply that every subsystem was deficient or that the underlying approach failed. They identify where engineering effort produced capabilities and where additional validation was still required, and they make those boundaries measurable rather than leaving them to a favorable or unfavorable overall impression.

TABLE X
REQUIREMENTS DERIVED FROM OBSERVED EVALUATION GAPS, WITH THE SECTION THAT EXHIBITS EACH FAILURE MODE.

Requirement	What to record	Failure mode it helps detect	Shown in
Identify the evaluated version	Software, configuration, input, and relevant environment state	Crediting a later capability to an earlier run	§XI-C
Preserve outcome categories	Completed answer, failure, interruption, timeout, and unresolved result	Turning process completion into diagnostic success	§IX
Establish fault truth separately	Injected condition, observation window, expected target, and restoration evidence	Treating a plausible explanation as confirmed cause	§VI
Test different evidence states	Positive, valid empty, rejected, unavailable, and ambiguous responses	Confusing lack of usable evidence with a negative finding	§V
Inspect relationships as well as facts	Support for target attribution and dependent observations	Attaching correct data to the wrong object	§X-A
Inspect the delivered artifact	What the operator actually received and what remained qualified	Grading an internal output that was not delivered	§IX
Supply the evaluator with raw returns	Full device responses or an independently grounded equivalent	A favorable grade inheriting the system's blind spot	§VII
Measure the full task interval	Receipt, waiting, investigation, delivery, review, and useful action	Presenting one fast stage as an operational speedup	§VIII

XIII. RELATED WORK

This section positions the findings against published work on language-model systems for operations, agent evaluation, model-based judging, grounding, and empirical method. It does not import results from other systems to support ours; each paragraph states what the cited work established and what this paper adds or leaves open.

A. Language-model systems for network and cloud operations

Cloud incident management has the most mature line of work. Ahmed et al. recommend root causes and mitigation steps from historical incident reports [1]; RCACopilot aggregates runtime diagnostic information per alert type, predicts a root-cause category, and reports accuracy on a year of internal Microsoft incidents [2]; Roy et al. examine tool-using agents for the same task [3]; and AIOpsLab supplies a framework that deploys microservice environments, injects faults, generates workload, and exports telemetry so that agents can be evaluated end to end [4]. In networking, NetAssistant is the closest deployed analogue: a task-oriented dialogue system that answers natural-language diagnosis questions in production data centers and reports reductions in on-call volume and time [5]. BiAn ranks error devices from monitoring data at production scale [6], Confucius applies multi-agent language models to intent-driven management [7], and NetLLM adapts language models to networking prediction and control tasks [8]. Configuration synthesis shows the same verification theme: Mondal et al. found GPT-4 alone produced configurations with errors in topology, syntax, and semantics and needed an external verifier in the loop [9], and NetConfEval benchmarks translation from requirements to specifications, API calls, algorithms, and low-level configuration [10]. Emulator-based troubleshooting arenas are now appearing [11].

Those systems are evaluated on aggregate accuracy, ranking quality, or on-call reduction over internal populations. This paper is narrower in scale and reports no such rate. What it adds is a decomposition those evaluations do not perform: the boundaries at which a command completion, an observed anomaly, a compressed context, a completion stamp, and a favorable grade each stopped being evidence. The probe experiment (Section V) also concerns a layer that emulator benchmarks generally hold fixed, since a benchmark that assumes accepted commands cannot expose a 19-of-256 rejection rate hidden under normal return. Industry analysts have defined agentic NetOps as a category and report adoption below one percent of organizations in early 2026 [31], [32]; the boundaries measured here are the kind of evidence such adoption decisions will need.

B. Evaluating tool-using agents

General agent benchmarks measure task completion in software engineering, web navigation, and tool-and-user interaction [12], [13], [14], [15]. Kapoor et al. argue that this practice over-weights accuracy, conflates the needs of model developers and downstream developers, lacks holdouts, and is poorly reproducible, and they propose cost-controlled and standardized evaluation [16]. Xia et al. analyze software-engineering agent trajectories rather than final outputs and find failures that output-only grading misses [17]. Our treatment of units of replication (Section IV-C) and the requirements in Table X follow the same direction in an operational setting where ground truth has to be planted and restored rather than read from a test suite. The completion-integrity results (Section IX) identify a failure class that output-graded benchmarks cannot see at all: harness fields asserting an answer while no delivered marker exists.

C. Model-based evaluation and its validity

Zheng et al. established that strong language models agree with human preference judgments at rates comparable to inter-human agreement while exhibiting position, verbosity, and self-enhancement biases [18]. Subsequent work quantified order sensitivity [19] and traced self-preference to self-recognition [20]. For agents, Xue et al. showed that judges given only a

final screenshot, or given every screenshot, agree poorly with humans, and that agreement depends on selecting the evidence the judge sees [21]; Agent-as-a-Judge evaluates trajectories with an agentic evaluator for the same reason [22]. These results characterize how a judge behaves given its evidence. The evaluator failure in Section VII is prior to that: the judge's input omitted the evidence, because a completion label had already replaced the device response. The success label alone does not identify command acceptance (Section IV-B); the historical evaluator's input omitted the full device returns, and it missed the retained rejections. Exact non-identifiability is established for the label-only projection, not for the evaluator's complete input, and Table X turns the observed gap into a requirement to supply raw returns.

D. Grounding, attribution, and context

Attribution research defines when generated text is supported by an identified source and measures how often systems fail that test [23], [24]. The target-binding term $B(e, k)$ in Section IV-A makes explicit an operational condition, that the source must concern the requested object, which attribution frameworks fold into source verification rather than record as a separate outcome. The field cases in Section X-A failed on that condition while the cited observation was itself accurate. Liu et al. showed that model accuracy degrades when relevant material sits in the middle of a long context [25], which is one reason summarized working context is used; Section X-C documents the cost, decision-relevant fields dropped before synthesis. In network diagnostics, differential provenance traced an observed difference to the events that caused it [26]; the retention and source-attribution work here is the agent-side counterpart without a formal provenance graph, and the formalization of that gap is left open.

E. Empirical method and performance measurement

The study follows case-study and experience-report guidance from empirical software engineering: stated context, a chain of evidence, purposive selection, and explicit validity threats [27], [28], [29], [30]. The latency result in Section VIII illustrates an Amdahl-style limit [33]: the gain in the collection and worker interval is diluted by intervals that did not improve. The post-worker interval was 36.62 of 81.74 seconds (44.80 percent) in L1 and 36.79 of 64.18 seconds (57.32 percent) in L3. The decomposition does not establish a fixed serial fraction or identify an internal causal bottleneck.

XIV. Validity, Disclosure, and Reproducibility

The study is retrospective. The census covers the declared retained local sources; the detailed adjudication covers specified experiments and selected audit cases, not every historical answer. Records are concentrated in approximately ten weeks within the longer engineering history and are uneven across time, software versions, and environments. Several campaigns lack immutable build and environment snapshots, some historical raw sources were unavailable for fresh review, and later documentation can revise an earlier interpretation. We preserve original execution populations when later corrected matrices lack corresponding rerun captures. The original customer correspondence and assessment material referenced by the internal post-mortem were not re-examined.

Cases outside the quantitative experiments were selected for engineering relevance, and Section IV-C states why none of the cohorts are independent trials. The numerical analysis does not estimate global diagnostic accuracy, independent fault-recovery probability, comparative model performance, expert equivalence, return on investment, or deployment-wide speedup. The MTU causal-review category N means no issue identified under that narrow rubric, not a certified correct answer; the declared criteria and per-case labels are supplied so an accountable engineer can review and revise them. Mechanical extraction enumerates identities, timestamps, markers, and explicit fields; it does not turn a keyword match into a diagnosis.

Historical grades, report fields, and completion indicators measure different things. Where they disagree with retained responses or delivery markers, the disagreement remains a finding. Present implementation may include later safeguards or dormant features that were not active during the recorded cases; static inspection and test definitions cannot establish runtime behavior in general, and where records report a test or a deployment, this paper credits the documented scope without turning that milestone into a broader guarantee.

The accompanying data tables permit reproduction of the reported arithmetic and consistency checks, including the measurements plotted in the figures, and a standard-library verification script recomputes 134 checks of arithmetic and cross-file consistency from those tables; passing establishes public-data arithmetic and cross-file consistency, not source authenticity, diagnostic correctness, human agreement, or disclosure clearance. The tables are curated measurement and adjudication extractions, not publicly released original transcripts, and a data dictionary states each field's meaning and missingness convention. A separate private ledger maps every evidence collection and selected claim to original files, hashes, and line or field handles. The public package therefore supports numerical reproducibility but not independent execution of the proprietary system or unrestricted forensic verification of the source records. An independent replication would require a separately authorized experimental arrangement, an appropriately cleared evidence set, and a frozen evaluation configuration. Author verification and confidentiality and intellectual-property clearance remain prerequisites to public release.

The principal unresolved empirical task is a controlled end-to-end evaluation with agreed fault ground truth, stable versions, positive and adverse-input cases, explicit treatment of interruptions, and measurement of operator outcomes. Larger-scale replay and laboratory work can support that effort, but planned scale tiers and available harness code are not evidence that the system has operated successfully at those scales. [E13]

XV. Conclusion

The retained engineering record supports a specific account of building and evaluating the troubleshooting capability of an agentic NetOps platform over nearly two years. It

includes hundreds of captured investigations, a fully reconciled command-probe population, a controlled configuration-state contrast, matched-prompt correction records, an event-based latency comparison, and audit-recorded field cases of target binding and evidence handling. These artifacts establish useful capabilities and show precisely where broader interpretations cease to be justified.

The central result is that execution success, evidence validity, claim support, diagnostic causality, evaluator verdicts, and operator delivery are different observables. A successful command is not necessarily usable evidence; a real anomaly is not necessarily the requested endpoint's problem; a summary is not necessarily an exhaustive record; a completion stamp is not necessarily a delivered answer; a favorable grade is not necessarily verified evidence. The engineering revisions and experiments are valuable because they expose those distinctions in actual system behavior. Maintaining them in evaluation makes both progress and remaining failures measurable, and supports neither dismissing the work because failures occurred nor declaring the remaining reliability questions resolved because individual repairs succeeded.

REFERENCES

[1] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang, and S. Rajmohan. Recommending root-cause and mitigation steps for cloud incidents using large language models. In *Proc. 45th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2023.

[2] Y. Chen, H. Xie, M. Ma, Y. Kang, X. Gao, L. Shi, Y. Cao, X. Gao, H. Fan, M. Wen, J. Zeng, S. Ghosh, X. Zhang, C. Zhang, Q. Lin, S. Rajmohan, D. Zhang, and T. Xu. Automatic root cause analysis via large language models for cloud incidents. In *Proc. 19th European Conference on Computer Systems (EuroSys)*, pages 674–688, 2024. doi:10.1145/3627703.3629553.

[3] D. Roy, X. Zhang, R. Bhave, C. Bansal, P. Las-Casas, R. Fonseca, and S. Rajmohan. Exploring LLM-based agents for root cause analysis. In *Companion Proc. 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion)*, pages 208–219, 2024.

[4] Y. Chen, M. Shetty, G. Somashekar, M. Ma, Y. Simmhan, J. Mace, C. Bansal, R. Wang, and S. Rajmohan. AIOpsLab: A holistic framework to evaluate AI agents for enabling autonomous clouds. In *Proc. Machine Learning and Systems (MLSys)*, 2025.

[5] H. Wang, A. Abhashkumar, C. Lin, T. Zhang, X. Gu, N. Ma, C. Wu, S. Liu, W. Zhou, Y. Dong, W. Jiang, and Y. Wang. NetAssistant: Dialogue based network diagnosis in data center networks. In *Proc. 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 2011–2024, 2024.

[6] C. Wang, X. Zhang, R. Lu, X. Lin, et al. Towards LLM-based failure localization in production-scale networks. In *Proc. ACM SIGCOMM 2025 Conference*, pages 496–511, 2025. doi:10.1145/3718958.3750505.

[7] Z. Wang, S. Lin, G. Yan, S. Ghorbani, M. Yu, J. Zhou, N. Hu, L. Baruah, S. Peters, S. Kamath, J. Yang, and Y. Zhang. Intent-driven network management with multi-agent LLMs: The Confucius framework. In *Proc. ACM SIGCOMM 2025 Conference*, pages 347–362, 2025. doi:10.1145/3718958.3750537.

[8] D. Wu, X. Wang, Y. Qiao, Z. Wang, J. Jiang, S. Cui, and F. Wang. NetLLM: Adapting large language models for networking. In *Proc. ACM SIGCOMM 2024 Conference*, pages 661–678, 2024. doi:10.1145/3651890.3672268.

[9] R. Mondal, A. Tang, R. Beckett, T. Millstein, and G. Varghese. What do LLMs need to synthesize correct router configurations? In *Proc. 22nd ACM Workshop on Hot Topics in Networks (HotNets)*, 2023. doi:10.1145/3626111.3628194.

[10] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa. NetConfEval: Can LLMs facilitate network configuration? *Proceedings of the ACM on Networking*, 2(CoNEXT2), Article 7, 2024. doi:10.1145/3656296.

[11] Z. Wang, A. Cornacchia, A. Sacco, F. Galante, M. Canini, and D. Jiang. A network arena for benchmarking AI agents on network troubleshooting. arXiv:2512.16381, 2025.

[12] X. Liu, H. Yu, H. Zhang, et al. AgentBench: Evaluating LLMs as agents. In *Proc. 12th International Conference on Learning Representations (ICLR)*, 2024.

[13] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proc. 12th International Conference on Learning Representations (ICLR)*, 2024.

[14] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig. WebArena: A realistic web environment for building autonomous agents. In *Proc. 12th International Conference on Learning Representations (ICLR)*, 2024.

[15] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *Proc. 13th International Conference on Learning Representations (ICLR)*, 2025.

[16] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir, and A. Narayanan. AI agents that matter. *Transactions on Machine Learning Research*, 2025.

[17] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang. Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE), 2025. doi:10.1145/3715754.

[18] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[19] P. Wang, L. Li, L. Chen, Z. Cai, D. Zhu, B. Lin, Y. Cao, L. Kong, Q. Liu, T. Liu, and Z. Sui. Large language models are not fair evaluators. In *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 9440–9450, 2024. doi:10.18653/v1/2024.acl-long.511.

[20] A. Panickssery, S. R. Bowman, and S. Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[21] T. Xue, W. Qi, T. Shi, C. H. Song, B. Gou, D. Song, H. Sun, and Y. Su. An illusion of progress? Assessing the current state of web agents. In *Proc. Conference on Language Modeling (COLM)*, 2025.

[22] M. Zhuge et al. Agent-as-a-Judge: Evaluate agents with agents. In *Proc. 42nd International Conference on Machine Learning (ICML)*, 2025.

[23] H. Rashkin, V. Nikolaev, M. Lamm, L. Aroyo, M. Collins, D. Das, S. Petrov, G. S. Tomar, I. Turc, and D. Reitter. Measuring attribution in natural language generation models. *Computational Linguistics*, 49(4):777–840, 2023.

[24] N. F. Liu, T. Zhang, and P. Liang. Evaluating verifiability in generative search engines. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 7001–7025, 2023.

[25] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[26] A. Chen, Y. Wu, A. Haeberlen, W. Zhou, and B. T. Loo. The good, the bad, and the differences: Better network diagnostics with differential provenance. In *Proc. ACM SIGCOMM 2016 Conference*, pages 115–128, 2016.

[27] P. Runeson and M. Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164, 2009. doi:10.1007/s10664-008-9102-8.

[28] P. Ralph, N. bin Ali, S. Baltes, D. Bianculli, J. Diaz, et al. Empirical standards for software engineering research. arXiv:2010.03525, 2020. Released as the ACM SIGSOFT Empirical Standards, 2021.

[29] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén. *Experimentation in Software Engineering*. Springer, 2012.

[30] R. Verdecchia, E. Engström, P. Lago, P. Runeson, and Q. Song. Threats to validity in software engineering research: A critical reflection. *Information and Software Technology*, 164:107329, 2023.

[31] J. Forest and A. Lerner. Agentic NetOps will revolutionize network operations. Gartner, Inc., research note, 6 August 2025.

[32] Gartner, Inc. Market guide for agentic NetOps software. Gartner, Inc., 19 May 2026.

[33] G. M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proc. AFIPS Spring Joint Computer Conference*, pages 483–485, 1967. doi:10.1145/1465482.1465560.

Original Evidence References

These references identify original project evidence, not external publications. Exact identifying source locations and hashes are retained only in the private ledger. Public CSVs use the same experiment aliases as the text. E1 to E7 correspond to the machine-censused and quantitatively reconstructed collections; E8 to E13 correspond to audit-recorded collections.

[E1] Retained-corpus census. Source manifests, identity and header extraction, invocation distinctions, driver-log exclusions, raw-output index, and historical capture coverage. Public tables: `corpus_captures.csv`; `measurements.json` (census reconciliation).

[E2] Direct platform-onboarding experiment. Original 260-cell record, retained nonempty device responses, rejected, valid-empty, and positive fixtures, later classification matrices, and inconsistent summary totals. Public table: `probe_matrix.csv`.

[E3] Controlled MTU campaign. Original manifest and execution script, baseline, change, and restoration log, eight pre-intervention and ten modified-state transcripts, and retrospective adjudication. Public tables: `mtu_case_timings.csv`, `mtu_output_lines.json`; review packet and rubric: `mtu_blind_review.json`; field definitions: `DATA_DICTIONARY.md`.

[E4] Rejection and evaluator study with staged correction. Sixty original transcripts and grades, seven matched-prompt reruns, separately retained targeted rejection and success checks, and evaluator implementation. Public table: `rejection_events.json`.

[E5] Execution-refinement sequence. Five original CLI transcripts, matched command identities, displayed durations, request, worker, and completion timestamps, and historical revision notes. Public tables: `all_calls.csv`, `matched_pairs.csv`, `lifecycle_event_timestamps.csv`, `precision_findings.json`.

[E6] Repeated-baseline and alert-result integrity. Ten fully joined baseline campaigns, three additional partial campaigns, 118 alert results, targeted delivery and attribution checks, and human-assisted review records. Public tables: `baseline_invocations.csv`, `alert_results.csv`; campaign aggregates in `measurements.json`; verification: `verify_supplement.py`.

[E7] Source history and longitudinal engineering records. Initial integrated implementation, representative multi-device, evidence, and interaction revisions, and the latest retained source snapshot. Source existence is distinguished from historically enabled runtime behavior.

[E8] External-evaluation forensic record. Task-by-task field investigations, positive observations, unsupported claims, partial results, interruptions, and open ground-truth questions.

[E9] Rehearsal and evidence-handling records. Summary completeness, source boundaries, time interpretation, and reported exact-detail recovery checks.

[E10] Later investigation and provenance audits. Unsupported intermediate associations, final-answer recovery, source-inspected checks, and narrowed validation claims.

[E11] Investigation continuation records. Discovery-dependent collection, unsuccessful intermediate approaches, later implementation, and bounded validation status.

[E12] Execution, interaction, and reliability records. Interactive and alert paths, permissions, safety tests, source attribution, harness runbooks, and terminal-outcome handling.

[E13] Broader readiness and replay records. Scale measurement plans, available harness components, fidelity limits, and unfinished validation work.

Appendix A
Protocol for a Controlled End-to-End Evaluation

This appendix specifies the study that Section XIV identifies as unresolved. It is a proposal derived from Table X and from the state-log discipline of Section VI; no part of it has been executed, and nothing here is evidence about the system.

a) Version and environment.: Freeze and record the build identifier, model identifiers and roles, configuration, and device software images before the first run. Record the environment class (physical, emulated, or mixed) and the known fidelity limits of any emulated component. Do not change any of these during the study; a change starts a new study.

b) Fault catalog.: Define each fault as a tuple: injection procedure, verification that the injection took effect, observation window, expected target object, restoration procedure, and verification of restoration. Cover at least one fault per evidence state in Figure 5a: a condition with a positive observation (for example a configuration inconsistency such as MTU), one whose lookup legitimately returns a valid empty result, one that provokes a rejected or incomplete command, one with an unavailable target, and an ambiguous-target case. Include no-fault control prompts. The fault injection, not the prompt, is the unit of replication; repeated prompts against one injection count once.

c) Arms.: Run at least two arms on the same fault catalog and prompt set: the system under test, and a contrast arm that removes or replaces one capability (for example evidence retention and source attribution) or substitutes a second model. Randomize run order within each fault and record it. Hold host, network, and model-service conditions as constant as the environment allows, and record what could not be held.

d) Prompts and adverse inputs.: Use symptom-language prompts that do not name the fault, drawn from the families in Table V. Add adverse inputs for each fault: an identifier in a representation the platform does not accept, a target that does not exist, and a prompt that presupposes a relationship the network does not have.

e) Outcome capture.: For every invocation record separately: process exit, completion stamp, delivered-artifact marker, and the artifact the operator would have received. Classify each invocation as completed answer, failure, interruption, timeout, or unresolved before any quality judgment is made. Retain every device return in full and classify each as positive, valid empty, rejected, incomplete, or unavailable at capture time.

f) Adjudication.: Two adjudicators, blinded to arm, label each delivered artifact on detection of the injected condition, target binding (the cited evidence concerns the expected target), causal premises (unsupported, ambiguous, or none), and whether a proposed verification was executed. Pre-register the rubric. Report agreement (for example Cohen's κ) and resolve disagreements by a recorded procedure. If an automated evaluator is also used, supply it with the full device returns and report its disagreement with the human labels as a result.

g) Timing.: Record t_0 to t_3 as in Equation (6), plus delivery time and human review time. Report elapsed intervals per stage; never sum stage durations that overlap.

h) Analysis and reporting.: Report every count with its denominator per fault injection and per arm. Report paired differences where the same fault and prompt were run in both arms. Do not pool arms, prompts, or repeated injections into one population, and do not report a confidence interval whose sampling model treats prompts as independent trials. Publish the fault catalog, prompt set, rubric, and per-invocation outcome table with the paper.